

Internet de las Cosas

Máster MUCNAP

Seminario 3

Tráfico inteligente en la Smart City

Versión 1.5

Actualización: 21/05/2024

Profesor: Joan Fons i Cors

Control de versiones

Fecha	Autor	Descripción
17/11/2020	Joan Fons	Versión Inicial completa del documento
30/11/2020	Joan Fons	Corrección de erratas
01/12/2020	Joan Fons	Cambiada ubicación Servicio a ttmi008.iot.upv.es
02/12/2020	Joan Fons	Añadidas secciones para simulación
19/11/2021	Joan Fons	Adaptaciones a curso actual
10/05/2022	Joan Fons	Adaptaciones al curso actual
21/05/2024	Joan Fons	Adaptaciones al curso actual

Tabla de Contenido

1	INTRODUCCIÓN	4
2	RED DE CARRETERAS: EL PARQUE TECNOLÓGICO DE PATERNA	6
2.1	Carreteras : Roads y RoadSegments	7
2.2	Red de Carreteras: Comunicación Directa con REST	8
3	VEHÍCULOS CONECTADOS	10
3.1	Vehículos: Comunicación Directa con REST	11
3.2	Vehículos : Navegación autónoma	13
3.3	Comunicación Indirecta Vehículos-Carreteras con MQTT	18
4	ESCENARIO SMART TRAFFIC	19
4.1	Diseño de las Comunicaciones con MQTT	19
4.2	INTERACCIÓN ENTRE CARRETERAS, VEHÍCULOS Y SEÑALES	21
4.2.1	LAS CARRETERAS DIFUNDEN SU ESTADO E INCIDENCIAS	21
4.2.2	LAS SEÑALES DE TRÁFICO PUBLICAN SU ESTADO	23
4.2.3	LOS VEHÍCULOS NOTIFICAN SU UBICACIÓN	24
4.2.4	SE PUBLICAN ALERTAS EN LAS CARRETERAS	26
5	EJERCICIOS A RESOLVER	27
6	EJERCICIO: SMART TRAFFIC REDUCIDO	28
6.1	Servicio de Smart Traffic Reducido	28
6.2	Ejercicio Paso 1. Iniciar la solución ...	30
6.3	Ejercicio Paso 2. Notificar Accidente por el Canal 'alerts'	36
6.4	Ejercicio Paso 3a. La Carretera Retransmite Alerta ...	39
6.5	Ejercicio Paso 3b. La SmartCity Envía una Ambulancia ...	41
6.6	Paso 4a) Los Vehículos Reciben Alerta ...	42
6.7	Ejercicio Opcional: Rutas	42
7	ANEXO - SIMULADOR SMARTTRAFFIC	43
7.1	Interactuar con el simulador vía REST	44
7.1.1	CONSULTAR EL ESTADO DEL SIMULADOR	44
7.1.2	¿QUÉ CARRETERAS TIENE NUESTRO SIMULADOR?	45
7.1.3	OBTENER INFORMACIÓN SOBRE UNA CARRETERA	46
7.1.4	OBTENER INFORMACIÓN SOBRE UN SEGMENTO DE CARRETERA	47
7.1.5	¿QUÉ VEHÍCULOS CIRCULAN POR EL SIMULADOR?	48

7.1.6	OBTENER INFORMACIÓN SOBRE UN VEHÍCULO CONCRETO	49
7.1.7	CREAR UN NUEVO VEHÍCULO EN EL SIMULADOR	49
7.1.8	ESTABLECER UNA RUTA A UN VEHÍCULO EXISTENTE	50
7.1.9	ELIMINAR UN VEHÍCULO DEL SIMULADOR	50
7.1.10	CONFIGURAR LA CREACIÓN DE VEHÍCULOS ALEATORIOS	50
7.1.11	CONFIGURAR LA CREACIÓN DE INCIDENTES ALEATORIOS	51
7.1.12	CONFIGURAR LA RESOLUCIÓN (ELIMINACIÓN) ALEATORIA DE INCIDENTES	53
7.1.13	CONSULTAR LOS TIPOS Y ROLES DE VEHÍCULOS PERMITIDOS	53

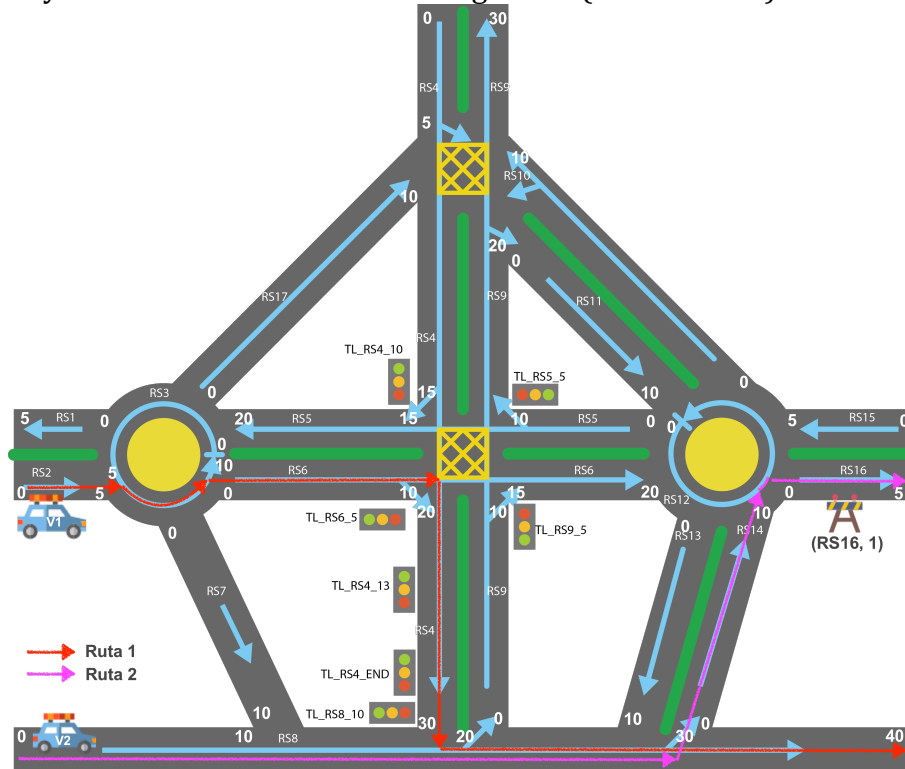
1 INTRODUCCIÓN

El escenario SmartTraffic propone un servicio que integra diferentes recursos y servicios en el ámbito del tráfico en las carreteras y ciudades, ofreciendo una plataforma que usa los conceptos de la Internet de las Cosas (Internet of Things, IoT) para integrar diferentes recursos. En este escenario se ofrecen los siguientes elementos/recursos/smart things, y está abierto a que se extienda con otros servicios y dispositivos:

- **Mapa o Red de carreteras:** formado por *roads*, que representan carreteras (o calles), que a su vez se definen por los segmentos que las componen (road segments). Estas carreteras proporcionan información sobre su estado (saturación de la vía, número de vehículos circulando actualmente, complicaciones, etc.)
- **Vehículos:** que pueden conectarse a la infraestructura vial para conocer el estado de las carreteras y las señales. Pueden además proporcionar información sobre incidentes (accidentes, averías, etc.)
- **Señales de Tráfico:** semáforos, límites de velocidad, etc. Cada señal de tráfico está posicionada en un RoadSegment, en un punto kilométrico concreto.
- **Incidencias:** que ocurren en las carreteras ('obras', 'accidentes', 'calles cortadas', etc.), provocando alguna complicación en el flujo de tráfico
- **SmartCity:** se puede ver como una entidad o recurso que gestiona y monitoriza los recursos que la componen (en este caso tráfico, vehículos, infraestructura, etc.) y tomar acciones buscando aplicar políticas de seguridad, de eficiencia energética, de aprovisionamiento de servicios, etc.
- **Otros servicios y dispositivos:** cualquiera puede añadir nuevos participantes e interactuar con las apis ya existentes, y ofrecer nuevas (por ejemplo, plazas de parking, estaciones de servicio/puntos de carga, contenedores de residuos, ...)
- **Un simulador:** para ayudar a reproducir escenarios realistas, este simulador es capaz de generar información sobre toda la infraestructura, pudiendo generar vehículos (virtuales que circulan por ella), señales de tráfico (por ejemplo, semáforos), incidencias, etc.

En este laboratorio veremos cómo diseñar una solución IoT (el controlador de un vehículo 'conectado' a la plataforma) y qué tipo de comunicaciones son necesarias, para permitir a los vehículos circular conectados a la infraestructura vial.

Nos centraremos en trabajar sobre la red de carreteras y sobre los vehículos. El simulador de tráfico estará operativo (queda fuera de los objetivos del laboratorio abordar cómo configurarlo). La SmartCity será un pequeño componente que desarrollaremos para desplegar la funcionalidad de coordinación entre las carreteras y el servicio de atención a emergencias (ambulancias).



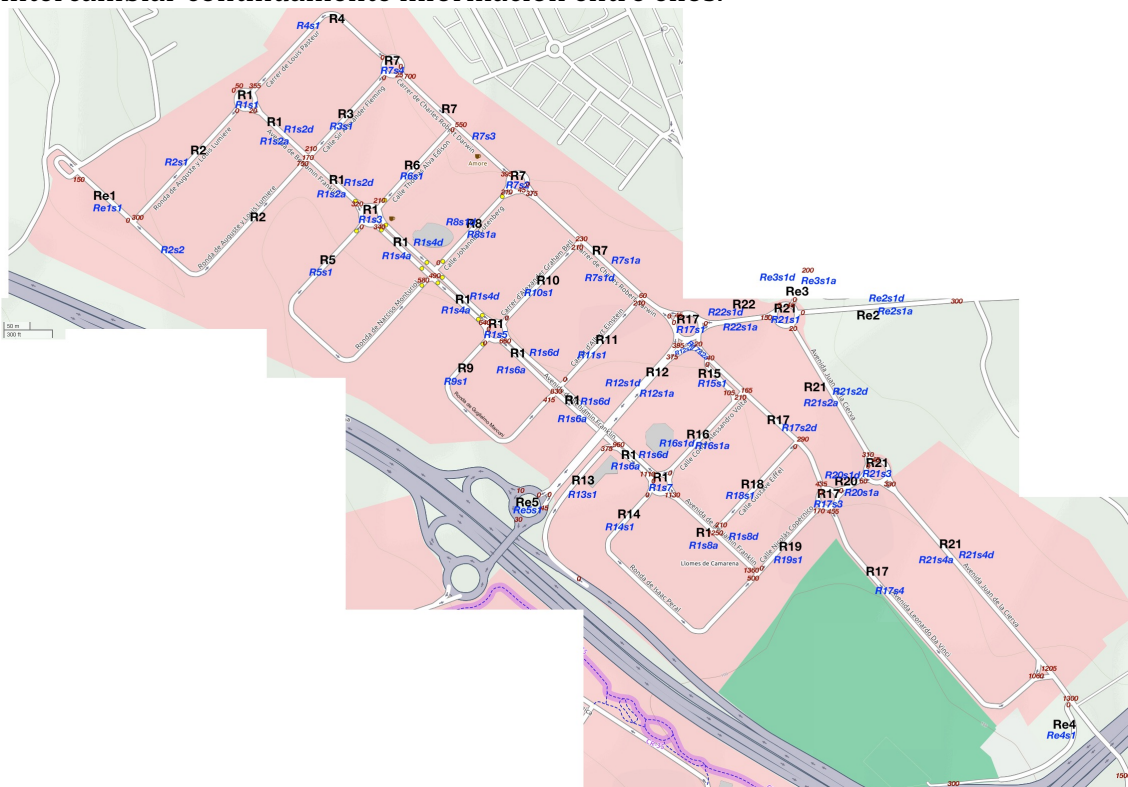
Este documento describe de manera detallada las relaciones y comunicaciones entre estos elementos y servicios, y cómo configurar un simulador para interactuar con ellos.

2 RED DE CARRETERAS: EL PARQUE TECNOLÓGICO DE PATERNA

Tal y como hemos comentado anteriormente deseamos crear un sistema IoT en el ámbito de las carreteras y ciudades inteligentes que otorgue una interacción con capacidades de inteligencia ambiental entre los diferentes participantes (vehículos, carreteras, señales, organizaciones, etc.) de manera inteligente, proactiva y sensible al contexto.

En este escenario se desea que las carreteras ofrezcan servicios para los vehículos que circulan en ella, de manera que en todo momento les mantengan informados sobre situaciones, incidencias o mensajes varios (publicidad, información sobre gasolineras, restaurantes, etc. en la carretera, ...).

En este laboratorio usaremos como infraestructura vial para mostrar los ejemplos el Parque Tecnológico de Paterna (PTPaterna). La Figura 1 muestra el ‘mapa de carreteras’ que vamos a usar. El diseño de esta infraestructura está pensada para permitir tanto a vehículos como a las propias carreteras mantenerse ‘conectados’ e intercambiar continuamente información entre ellos.



2.1 CARRETERAS : ROADS Y ROADSEGMENTS

En el mapa se pueden observar, en color negro los nombres de las carreteras/calles (de R1 a R22), y en azul los diferentes 'segmentos' o tramos de estas calles (R5s1 ó R17s4, por ejemplo). En rojo aparecen las distancias (expresadas en metros) dentro de cada segmento.

Una carretera se define como una secuencia de segmentos, cada uno de los cuales tienen un punto de inicio (start-kp) y un punto de fin (end-kp). Por ejemplo, la carretera (road) R2 se compone de los segmentos: R2s1 y R2s2. El segmento R2s1 tiene una longitud de 300 m (start-kp=0, end-kp=300), y el R2s2 una longitud de 450 m (start-kp=300, end-kp=750).

En cualquier punto de un segmento puede existir una intersección que lo une con otro segmento. Así, por ejemplo, para indicar que el segmento R2s1 termina o continua con el segmento R2s2, hay una intersección entre ambos: { (R2s1, 300) – (R2s2, 300) }.

Esta red de carreteras está disponible en los siguientes servidores:

<http://ttmi008.iot.upv.es:8182>

ó

<http://tambori.dsic.upv.es:10082/>

Puede usarse indistintamente uno u otro para realizar el trabajo. En este documento se usará el servicio desplegado en el nodo ttmi008.iot.upv.es (requiere estar en la UPV ó con la VPN-UPV), pero serviría igualmente para tambori.dsic.upv.es:10082 (se puede acceder desde fuera de la UPV).

Esta red de carreteras nos ofrece dos mecanismos para interactuar con ella:

- Una comunicación directa, a través de una API REST, que nos permite consultar en cada momento de la situación de alguno de los recursos que la componen
- Una comunicación indirecta, a través de colas de mensajería implementadas con MQTT, en la que la red de carreteras avisa/publica (a través del envío de mensajes) cuando ocurra algún evento de interés (un cambio en el estado de la carretera, un incidente, etc.). Dependiendo del servicio que utilicemos, deberemos interactuar a través de un bróker MQTT u otro:
 - Red de Carreteras en ttmi008 -> tcp://ttmi008.iot.upv.es:1883
 - Red de Carreteras en tambori -> tcp://tambori.dsic.upv.es:10083

Primero mostraremos el API REST para consultar el estado de las carreteras, y dejaremos la explicación de la comunicación indirecta para usar este mecanismo como medio de interacción entre carreteras y vehículos mediante MQTT.

2.2 RED DE CARRETERAS: COMUNICACIÓN DIRECTA CON REST

A continuación se describe el subconjunto del API REST que ofrece la red de carreteras para consultar información sobre sus recursos principales (carreteras, segmentos de carretera y vehículos). Para consultarla, podéis usar un cliente REST (RESTClient, Postman, o similar), o desde código Java (como ya hicimos en el laboratorio dedicado a REST).

Las representaciones que devuelven los recursos de la red de carreteras, contienen la siguiente información (según el recurso):

rt :	'resource type' ó tipo de recurso (road, road-segment, traffic signal, etc.)
name :	nombre de la carretera
id:	identificador de la carretera
segments:	tramos que conforman la carretera
start-kp y end-kp:	puntos kilométricos inicial y final
density:	saturación del tramo de carretera (# vehículos / longitud tramo)
capacity:	capacidad máxima (absoluta) en la carretera/segmento. Indica los vehículos que 'caben' circulando antes de colapsar.
num-vehicles:	número de vehículos circulando por el segmento
max-speed:	máxima velocidad permitida en este tipo de vía
current-max-speed:	máxima velocidad permitida 'en estos momentos'
status:	saturación del segmento (Free Flow, Limited Manouvers, Collapsed, Closed)

API REST DE CARRETERAS:

1. Obtener información sobre todas las carreteras disponibles

GET, <http://ttmi008.iot.upv.es:8182/roads>

Ejemplo de respuesta:

```
[{"rt":"road","link":"/road/R1","name":"Avenida de Benjamin Franklin",  
"id":"R1","segments":[{"rt":"road-segment","end-kp":50,"code":"R1s1",  
"density":0,"link":"/segment/R1s1","length":50,"max-speed":40,"road-
```

```
segment":"R1s1","start-kp":0,"capacity":6,"current-max-speed":40,  
"road":"R1","num-vehicles":0,"status":"Free_Flow"},  
...  
{ "rt":"road","link":"/road/Re5","name":"Re5 Rotonda  
Acces","id":"Re5","segments":[{"rt":"road-segment","end-  
kp":50,"code":"Re5s1","density":0,"link":"/segment/Re5s1","length":50,"max-  
speed":50,"road-segment":"Re5s1","start-kp":0,"capacity":6,"current-max-  
speed":50,"road":"Re5","num-vehicles":0,"status":"Free_Flow"}]}}
```

Como se puede observar, esta llamada fue realizada en un momento en el que no había ningún vehículo circulando por la red de carreteras (la densidad y número de vehículos están a 0, y el estado es 'Free_Flow').

2. Obtener información sobre una carretera (ej: R10)

GET, <http://ttmi008.iot.upv.es:8182/road/R10>

Ejemplo de respuesta:

```
{ "rt":"road","link":"/road/R10","name":"Carrer d'Alexander Graham Bell",  
"id":"R10","segments":[{"rt":"road-segment","end-kp":210,"code":"R10s1",  
"density":0,"link":"/segment/R10s1","length":210,"max-speed":60,"road-  
segment":"R10s1","start-kp":0,"capacity":21,"current-max-speed":60,  
"road":"R10","num-vehicles":0,"status":"Free_Flow"}]}}
```

3. Obtener información sobre un segmento concreto de una carretera (ej: R5s1)

GET, <http://ttmi008.iot.upv.es:8182/segment/R5s1>

Os proponemos a continuación un sencillo ejercicio:

Ejercicio 1: Lanzar las consultas anteriores y contrastar los resultados. ¿Qué longitud tiene el segmento R5s1?

También podemos acceder al listado de vehículos simulados que se pueden crear con el simulador (inicialmente está vacío) a través del recurso 'vehicles'

4. Obtener los vehículos simulados en ejecución

GET, <http://ttmi008.iot.upv.es:8182/vehicles>

3 VEHÍCULOS CONECTADOS

Para llevar a cabo el laboratorio, se tendrá acceso a una flota de vehículos inteligentes que son capaces de interactuar/comunicarse con las carreteras, sus señales de tráfico y las incidencias, y pueden seguir rutas de navegación. También se pueden controlar remotamente estos vehículos.

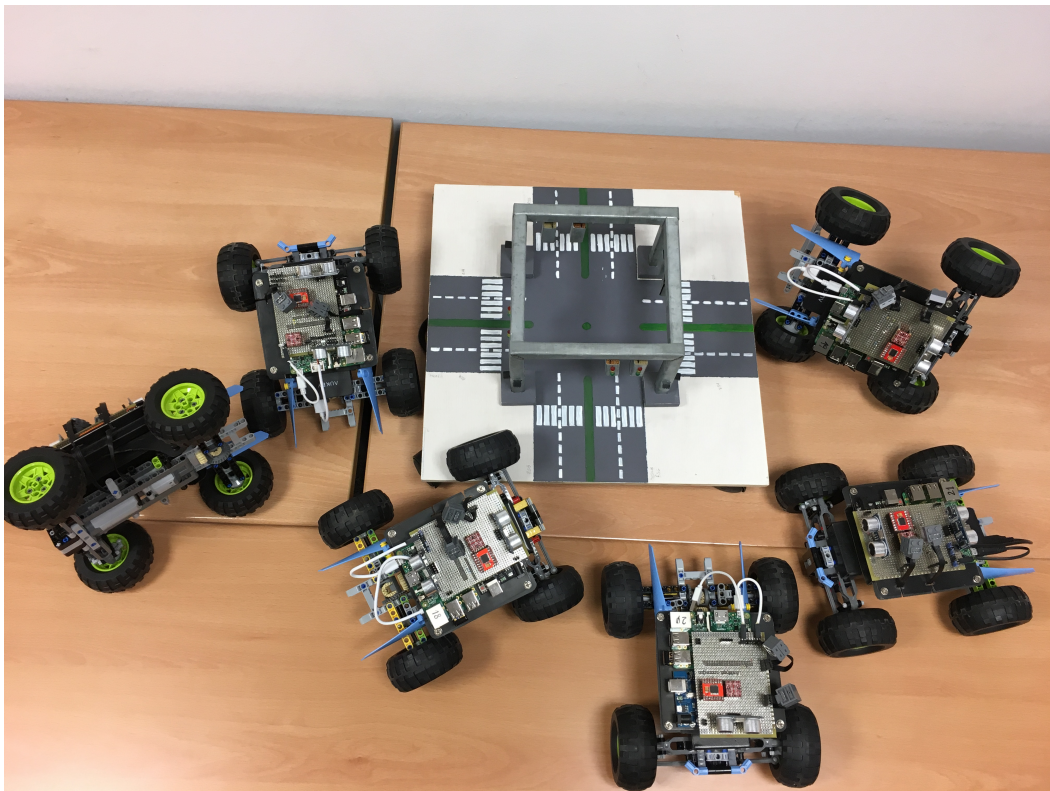


Figura 2: Maquetas de vehículos conectados

Cada vehículo se identifica con un número (del 16 al 21) y es accesible vía su URI. Estas URIs son sólo accesibles dentro de la UPV, a través de la red UPV-IoT (es necesario que estés conectado a la red UPVNET/UPVNET2G o UPV-IoT para poder acceder a ellos). Por ejemplo, los vehículos 18y 21 son accesibles en sus URIs:

```
ttmi018.iot.upv.es  
ttmi021.iot.upv.es
```

3.1 VEHÍCULOS: COMUNICACIÓN DIRECTA CON REST

Todos los vehículos ofrecen una API REST para interactuar directamente con ellos. A continuación se describen las llamadas más interesantes para el objetivo de este laboratorio. Vamos a asumir que trabajamos con el vehículo 21.

API REST VEHÍCULOS:

1.- Obtener información sobre el vehículo, y sus sensores

GET, <http://ttmi021.iot.upv.es:8182/car>

Ejemplo de respuesta:

```
{
  "id": "SmartCar021",
  "cruiser-speed": 70,
  "speed": 0,
  "status": "Stopped",
  "engine": {
    "device-type": "engine",
    "id": "engine",
    "movement": "stopped",
    "speed": 0
  },
  "distance-sensor": [
    {
      "distance": 17,
      "id": "fdistance"
    },
    {
      "distance": 30,
      "id": "rdistance"
    }
  ],
  "navigator": {
    "id": "SmartCar021_navigator",
    "status": "WAITING"
  },
  "steering": {
    "device-type": "steering",
    "id": "steering",
    "position": 0,
    "status": "CENTERED"
  }
}
```

Ejercicio 2: ¿Qué dispositivos nos ofrece el vehículo? Busca los ids de los dispositivos que tiene el vehículo (según su descripción).

Ejercicio 3: Consultar la distancia frontal sobre un vehículo, y ver qué ocurre cuando existe un obstáculo delante.

3.- Controlar el motor (tracción) del vehículo. Ofrece la llamada

PUT, <http://ttmi021.iot.upv.es:8182/car/engine>

con un Payload (en formato JSON)

```
{ "action" : <action-value>, "speed": <speed> }
```

siendo:

<action-value> uno de los siguientes valores: forward, brake, reverse.

<speed> un valor entre 0 y 100 (este valor es opcional, toma por defecto 100)

Para que esta llamada funcione, hay que añadir la siguiente cabecera HTTP. Si estás usando un cliente REST, hay que añadir un Header:

```
"Content-Type" -> "application/json"
```

Os proponemos a continuación un sencillo ejercicio:

Ejercicio 4: Probar a controlar el motor del vehículo¹

4.- Controlar la dirección. Ofrece la llamada

PUT, <http://ttmi021.iot.upv.es:8182/steering>

con un Payload (en formato JSON)

```
{ "action" : <action-value>, "position": <pos> }
```

siendo:

<action-value> uno de los siguientes valores: left, center, right

<pos> un valor entre 0 y 6 (este valor es opcional, toma por defecto 6).

Para que esta llamada funcione, hay que añadir la siguiente cabecera HTTP. Si estás usando un cliente REST, hay que añadir un Header:

```
"Content-Type" -> "application/json"
```

Os proponemos a continuación un sencillo ejercicio:

Ejercicio 5: Probar a controlar la dirección del vehículo.

¹ Si es el primer comando que recibe el coche, puede que no atienda a la petición (debido a un problema con el búffer de comunicaciones de Arduino). Para solucionarlo, enviar diferentes comandos (a partir del segundo debería funcionar sin problemas).

3.2 VEHÍCULOS : NAVEGACIÓN AUTÓNOMA

Una de las funcionalidades que nos ofrecen los vehículos es la de seguir una ruta de navegación. Incluyen un componente 'navigator' al que se le puede configurar una ruta de navegación. Al establecer esta ruta (indicada sobre un mapa de carreteras), el vehículo la cual seguirá, e interactuará con las carreteras a medida esté circulando, indicando en todo momento por dónde circula (en qué punto kilométrico de qué carretera), o incidiendo si va 'entrando' o 'saliendo' de alguna carretera. Así, por ejemplo, cada cierto tiempo o metros recorridos, el vehículo reportará que está (por ejemplo) en el segmento R2s1, en el punto kilométrico 50. Cuando deje este segmento y entre en el siguiente (R2s2), reportará dos mensajes: 1) he salido de R2s1, y 2) he entrado en R2s2 (estos mensajes se definen más adelante)-.

Una **ruta de navegación**, o simplemente una ruta, se identifica por una secuencia de '**tramos**', que definen secciones o 'tramos' de una carretera por la que se debe mover. Estos tramos, a su vez, están formados por un par de '**puntos**' (inicial y final) de este tramo, siendo el inicial el punto kilométrico en el que el vehículo 'entra' a dicha carretera, y el final el punto kilométrico en el que el vehículo 'sale' de dicha carretera (para entrar en el siguiente tramo). Tiene un punto de inicio (donde se ubicará el vehículo al empezar la ruta) y un punto de fin (en el que se quedará 'aparcado' al terminar).

Los puntos se definen con la siguiente estructura:

```
{"road":<id-segmento>,"point":<punto-kilometrico>}
```

Ejemplo: Punto kilométrico 0 del segmento R1S1

```
{"road":"R1S1","point":0}
```

Los tramos se definen como una pareja (ordenada) de puntos, del mismo segmento de carretera:

```
[ {"road":<id-segmento>,"point":<punto-kilometrico-inicio-tramo>},  
  {"road":<id-segmento>,"point":<punto-kilometrico-fin-tramo>} ]
```

Ejemplo: Tramo entre el punto kilométrico 0 y 20 del segmento R1S1

```
[{"road":"R1S1","point":0},{ "road":"R1S1","point":20}]
```

Y finalmente, una ruta se define como una sucesión de tramos que deberán 'navegarse' secuencialmente:

```
{ "route" : [  
  [{"road":<id-segmento>,"point":<pk-inicio>},{ "road":<id-segmento>,"point":<pk-fin>}],  
  ...  
  [{"road":<id-segmento>,"point":<pk-inicio>},{ "road":<id-segmento>,"point":<pk-fin>}]  
]}
```

Por ejemplo, la siguiente ruta define una circulación entre el punto kilométrico 0 del segmento R1S1 (inicio de la ruta), por la que seguiremos hasta el punto kilométrico 20 de dicho segmento. Luego nos incorporamos al punto kilométrico 60 de la R1S2 por la que seguiremos hasta el punto kilométrico 320, y de ahí cambiaremos de carretera al punto kilométrico 0 de R5S1 por la que seguiremos hasta detenernos en el punto 300 de dicha carretera. Comprobar en el mapa de carreteras del Parque Tecnológico, de dónde a dónde va el vehículo.

```
{ "route": [  
  [{"road": "R1S1", "point": 0}, {"road": "R1S1", "point": 20}],  
  [{"road": "R1S2a", "point": 0}, {"road": "R1S2a", "point": 320}],  
  [{"road": "R1S3", "point": 0}, {"road": "R1S3", "point": 10}],  
  [{"road": "R5S1", "point": 0}, {"road": "R5S1", "point": 300}]  
]
```

Cambiar/Establecer la ruta del vehículo

Usando esta notación, es posible consultar y establecer la ruta de un vehículo. Esto se puede aplicar a los vehículos físicos y a los vehículos virtuales (aquellos que circulan 'dentro' del simulador). Ambos tipos de vehículos pueden interactuar entre sí a través de la infraestructura IoT (APIs REST y MQTT).

El API rest expone el recurso 'navigator':

<http://ttmi021.iot.upv.es:8182/navigator>

o para un vehículo 'virtual-simulado'²

<http://ttmi008.iot.upv.es:8182/vehicle/3241JVM/navigator>

Por ejemplo, para consultar la situación de navegación, debemos ejecutar:

GET, <http://ttmi008.iot.upv.es:8182/vehicle/3241JVM/navigator>

² Podemos crear vehículos virtuales manual/explicitamente en el simulador usando el API de configuración del Simulador. También podemos configurar el simulador para que cree vehículos aleatoriamente, con rutas también aleatorias. Ver Anexo de este documento donde se describen las principales operaciones del API que usaremos para interactuar y configurar el simulador.

que cuando no hay ninguna ruta, nos devolverá que el estado del navegador es 'esperando ruta' ...

```
{  
  "id": "3241JVM_navigator",  
  "status": "WAITING"  
}
```

Si se desea establecer una ruta para un vehículo, deberá enviarse a través del recurso 'navigator'. El siguiente payload de ejemplo cambia la ruta al vehículo anterior:

```
{ "route": [[{"road": "R1S1", "point": 0}, {"road": "R1S1", "point": 20}], [{"road": "R1S2", "point": 20}, {"road": "R1S2", "point": 320}], [{"road": "R5S1", "point": 0}, {"road": "R5S1", "point": 300}]] }
```

*NOTA: recordar indicar las cabeceras "Content-Type": "application/json" y "Accept": "application/json"

y tendrá que enviarse a:

PUT, <http://ttmi008.iot.upv.es:8182/vehicle/3241JVM/navigator>

Tras enviar esta llamada, comprobar el estado (su ubicación) del vehículo lanzando una consulta ...

GET, <http://ttmi021.iot.upv.es:8182/car>

ó sobre un vehículo del simulador (este vehículo existirá si el simulador lo ha creado aleatoriamente, o si se lo hemos creado nosotros 'manualmente' usando el API de configuración del simulador)

GET, <http://ttmi008.iot.upv.es:8182/vehicle/3241JVM>

Si realizamos varias llamadas, podemos ver cómo progresa el vehículo en su ruta. En este primer paso, el vehículo (un autobús) se está moviendo a 80Km/h, y está (current-position) en el primer tramo, en el segmento RS1, punto kilométrico 660. Se puede observar la ruta pendiente en 'route' y la distancia (que calcula) hasta el destino (1040 m). La ruta que sigue le llevará de (RS1,0) a (RS3,250).

```
{
  "navigator": {
    "off-road": true,
    "route": "[ (RS1,0)-(RS1,1000)] [(RS2,50)-(RS2,500)] [(RS3,0)-(RS3,250)]",
    "current-position": "(RS1,660)",
    "destination": "(RS3,250)",
    "remaining-distance": 1040,
    "id": "3241JVM_navigator",
    "status": "ROUTING"
  },
  "id": "3241JVM",
  "characterization": { "role": "PublicTransport", "type": "Bus" },
  "cruiser-speed": 80,
  "speed": 80,
  "status": "Moving"
}
```

En otro momento el vehículo se puede observar que se encuentra en el segundo tramo (RS2 y punto kilométrico 370), la ruta restante y la distancia al destino (380m).

```
{
  "navigator": {
    "off-road": true,
    "route": "[ (RS2,50)-(RS2,500)] [(RS3,0)-(RS3,250)]",
    "current-position": "(RS2,370)",
    "destination": "(RS3,250)",
    "remaining-distance": 380,
    "id": "3241JVM_navigator",
    "status": "ROUTING"
  },
  "id": "3241JVM",
  "characterization": { "role": "PublicTransport", "type": "Bus" },
  "cruiser-speed": 80,
  "speed": 80,
  "status": "Moving"
}
```

Cuando termina, elimina se queda 'por defecto' aparcado en el punto final de último tramo de la ruta (RS3,250).

```
{  
  "navigator": {  
    "off-road": false,  
    "route": "<No route>",  
    "current-position": "(RS3,250)",  
    "destination": "(RS3,250)",  
    "remaining-distance": 0,  
    "id": "3241JVM_navigator",  
    "status": "REACHED_DESTINATION"  
  },  
  "id": "3241JVM",  
  "characterization": { "role": "PublicTransport", "type": "Bus" },  
  "cruiser-speed": 80,  
  "speed": 0,  
  "status": "Parked"  
}
```

3.3 COMUNICACIÓN INDIRECTA VEHÍCULOS-CARRETERAS CON MQTT

Hasta ahora hemos visto cómo interactuar y controlar ‘directamente’ los recursos de la plataforma. En esta comunicación, la interacción con estos recursos es directa (tipo *pull* / síncrona) en una relación ‘un recurso interactúa con otro/un recurso’ en cada interacción.

Vamos a ver ahora, sin embargo, otro mecanismo de comunicación en el que los vehículos y las carreteras van a intercambiar información, pero de una manera indirecta y de tipo *push* / asíncrona. En esta comunicación va a existir un intermediario (middleware de comunicación) que se encargará de canalizar y distribuir los mensajes a múltiples recursos, de manera que se va a permitir que ‘un recurso se comunique (publique) con múltiples recursos (suscriptores)’.

IMPORTANTE: Además, en este tipo de comunicaciones, es factible que tanto publicadores como suscriptores cambien dinámicamente (no siempre sean los mismos participantes, e incluso que se inviertan los papeles). También se asume un funcionamiento ‘push’ de comunicaciones, de manera que no es necesario que el suscriptor esté continuamente preguntando si ha habido cambios, ya que se notificará automáticamente cuando alguna comunicación de interés (suscrita) ocurra.

En el caso de las carreteras y los vehículos, son varios los escenarios en los que es interesante aplicar este patrón de comunicación:

- una carretera quiere informar a todos los vehículos que circulan por ellas que ha habido una retención, obras en un carril, o un incidente
- las señales de tráfico pueden informar de cambios en su estado (p.ej., semáforos o límites de velocidad)
- un vehículo quiere comunicar (difundir) a la red de carreteras que ha sufrido un accidente

4 ESCENARIO SMART TRAFFIC

Tomando como referencia la infraestructura que hemos proporcionado vamos a desarrollar un escenario de **'SmartTraffic'**.

Imaginemos que un vehículo circula por una carretera (por ejemplo, circula por *R1s4a* en punto kilométrico *600*). Al sufrir un accidente, informará de éste hecho automáticamente por un **canal de alertas**. La carretera inmediatamente retransmitirá este hecho por un **canal de información** para que otros vehículos que circulen por esta misma carretera sean conscientes. Además, la **ciudad** que es responsable de gestionar esta carretera, también **recibirá la alerta y enviará una ambulancia** al lugar del incidente.

4.1 DISEÑO DE LAS COMUNICACIONES CON MQTT

Para que este modelo de comunicaciones funcione, debe establecerse un **protocolo de comunicación M2M** (a través de un conjunto de colas de mensajería) que se usarán como canales de comunicación. Sobre estos canales, unos recursos publicarán contenidos, que se difundirán a aquellos recursos que se han suscrito a sobre estos mismos canales.

Las suscripciones a estos canales es algo dinámico, pudiendo cambiar con el tiempo (según las circunstancias). De esta manera, dos acciones de publicación sobre un mismo topic no tiene porqué notificar a los mismos suscriptores (si estos han cambiado). *Así, por ejemplo, en el caso de las carreteras y vehículos conectados, los vehículos se irán suscribiendo a los canales de información que ofrece el segmento de carretera por el que circulan en este momento, y cancelarán su suscripción a éstos cada vez que cambien de carretera.*

En MQTT, estos canales de comunicación se llaman **topics**, y la estructura que usaremos en este laboratorio para referenciarlos es la siguiente:

`<topic-base>/road/{road-id}/<canal-información>`

dónde:

`<topic-base>` prefijo (es/upv/pros/tatami/smartcities/traffic/PTPaterna) para indicar la solución que estamos usando (servicio/simulador para P.T. de Paterna)
`{road-id}` será el tramo de carretera por la que vamos
`<canal-información>` será el 'canal' de información al que nos suscribimos o al que publicamos. Usaremos los canales: **info**, **signals**, **traffic** y **alerts**.

Así, por ejemplo, la información sobre incidencias (retenciones, accidentes, etc.) en el segmento R5s1, las publica la red de carreteras y se suscriben todos aquellos vehículos que circulan actualmente por este tramo en el topic:

`es/upv/pros/tatami/smartcities/traffic/PTPaterna/road/R5s1/info`

Por otro lado, si un vehículo quiere informar de un accidente que ha sufrido en un tramo de carretera (por ej. en R5s1), tendrá que publicar en el topic:

`es/upv/pros/tatami/smartcities/traffic/PTPaterna/road/R5s1/alerts`

indicando la ubicación exacta (punto kilométrico) del accidente. A este topic de `alerts` estará suscrito tanto la carretera (para retransmitir al resto de vehículos por el topic `info`), como la smart city (para enviar una ambulancia al siniestro).

La siguiente figura resume los participantes y los diferentes canales de comunicación de este servicio de tráfico inteligente:

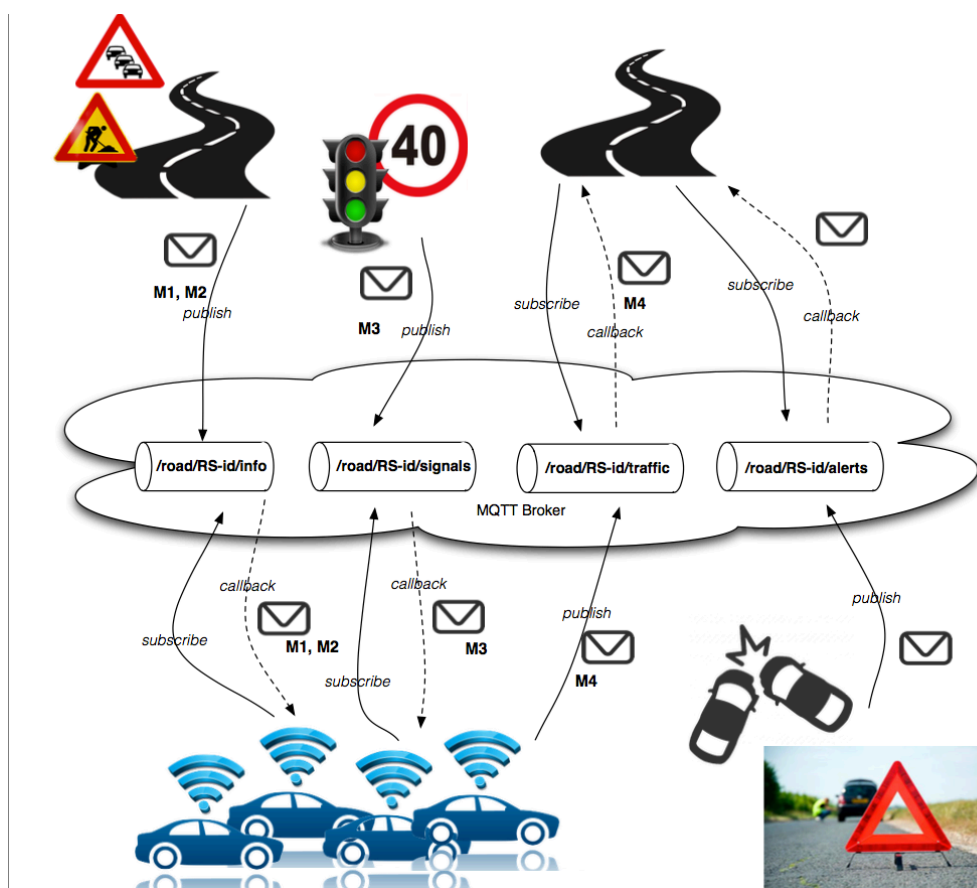


Figura 3 - Esquema de Comunicaciones indirectas en el servicio Smart Traffic

A continuació se detallen las interacciones que ya están desarrolladas y funcionales, y que usaremos para desarrollar nuevos escenarios/ejercicios.

4.2 INTERACCIÓN ENTRE CARRETERAS, VEHÍCULOS Y SEÑALES

4.2.1 LAS CARRETERAS DIFUNDEN SU ESTADO E INCIDENCIAS

Las carreteras publican en el topic

```
<topic-base>/road/{id-segmento}/info
```

mensajes M_1 y M_2 sobre incidencias ocurridas en cada tramo de carretera (el {id-segmento} es el identificador de RoadSegment, por ejemplo R1S1).

Por este canal las carreteras difunden/publicarán los accidentes, obras, etc. (incidencias), pero también el estado de la carretera (road status) en el que indicará, entre otros, el grado de saturación de la carretera (que se modificará dinámicamente a medida que entren vehículos en ella y no salgan).

A este canal se deben suscribir los vehículos al entrar en un road-segment, al seguir su ruta de navegación, y deberán des-suscribe al salir del segmento.

Ejemplo de mensaje M1: Estado Carretera (RoadSegment Status)

Topic MQTT: <topic-base>/road/{id-segmento}/info

```
{
  "id":MSG_1477474100254,
  "type":"ROAD_STATUS",
  "timestamp":1477474100254,
  "msg":{
    "rt":"road-segment",
    "road":"R11",
    "road-segment":"R11S3",
    "start-kp":0,
    "end-kp":377,
    "length":377,
    "capacity":8,
    "num-vehicles":2,
    "density":25,
    "status":"Mostly_Free_Flow",
    "max-speed":30,
    "current-max-speed":30,
    "code":"1",
    "link":"/segment/R11S3"
  }
}
```

Ejemplo de mensaje M2. Incidencia en Carretera (RoadIncident)

Topic MQTT: <topic-base>/road/{id-segmento}/info

```
{
  "id":"MSG_1477472669651",
  "type":"ROAD_INCIDENT",
  "timestamp":1477472669651,
  "msg":{
    "rt":"traffic::incident",
    "incident-type":"INCIDENT",
    "id":"712963150",
    "road":"R8",
    "road-segment":"R8S1",
    "starting-position":584,
    "ending-position":684,
    "description":"Working Area",
    "status":"Active",
    "link":"/incident/712963150"
  }
}
```

4.2.2 LAS SEÑALES DE TRÁFICO PUBLICAN SU ESTADO

Las señales de tráfico publican en el topic

`<topic-base>/road/{id-segmento}/signals`

mensajes M3 con el estado de la señal. Cada señal publica información en el `{id-segmento}` en el que se ubica dicha señal. De esta manera, cualquier vehículo podrá 'filtrar' y sólo recibir información sobre las señales que afectan el tramo por el que circula el vehículo.

Si es un semáforo (`signal-type=TRAFFIC_LIGHT`), indicará si está en rojo (`HLL`), amarillo (`LHL`) o en verde (`LLH`), y si es un límite de velocidad (`signal-type=SPEED_LIMIT`), indicará el (nuevo) límite.

A este canal se debe suscribir cada vehículo que entra en un road-segment, al seguir su ruta de navegación con el objetivo de cumplir siempre la normativa (las señales pueden cambiar en cualquier momento), y se des-suscribe al salir de éste.

Ejemplo de mensaje M3. Señal de Tráfico (Estado Semáforo)

Topic MQTT: `<topic-base>/road/{id-segmento}/signals`

```
{
  "id": "MSG_1477473769511",
  "type": "TRAFFIC_SIGNAL",
  "timestamp": 1477473769511,
  "msg": {
    "rt": "traffic-signal",
    "id": "TL_atR1S3_542",
    "road": "R1",
    "road-segment": "R1S3",
    "signal-type": "TRAFFIC_LIGHT",
    "starting-position": 542,
    "ending-position": 542,
    "value": "HLL"
  }
}
```

4.2.3 LOS VEHÍCULOS NOTIFICAN SU UBICACIÓN

Los vehículos publican en el topic

```
<topic-base>/road/{id-segmento}/traffic
```

mensajes M4 indicando que han entrado o han salido de este road segment ({id-segmento}) en concreto.

Cuando un vehículo accede (entra) en un segmento de carretera, debe reportar, con un mensaje 'VEHICLE_IN', que está en dicho segmento.

A medida que van avanzando por la carretera, los vehículos pueden reportar su situación actual con mensajes 'CHECK_IN'. En realidad sólo es 'necesario' enviar estos 'check-ins' para los vehículos especiales (ambulancias, policía, bomberos, etc.) con el objetivo que los otros vehículos sean conscientes por dónde circulan y puedan ceder el paso adecuadamente.

Cuando un vehículo abandona un segmento de carretera (sale), debe reportar, con un mensaje 'VEHICLE_OUT', que ya no está en dicho segmento.

A este canal se suscriben las carreteras (nunca se des-suscriben), y usan estos mensajes M4 ('VEHICLE_IN', 'VEHICLE_OUT' y 'CHECK_IN') para estimar en todo momento cuántos vehículos circulan por el tramo. Esta información la utilizan para saber su nivel de congestión, y en caso de cambiar de estado (de fluído a denso, por ejemplo), lo reportarán con un mensaje M1 en el topic:

```
<topic-base>/road/{id-segmento}/info
```

Ejemplos de mensajes M4. Entrada/Salida/Check-In Vehículo en tramo carretera

Topic MQTT: `<topic-base>/road/{id-segmento}/traffic`

Ejemplo de Mensaje 'VEHICLE_IN' al entrar en un Road Segment:

```
{
  "id": "MSG_1477473530870",
  "type": "TRAFFIC",
  "timestamp": 1477473530870,
  "msg": {
    "action": "VEHICLE_IN",
    "road": "R1",
    "road-segment": "R1S4a",
    "vehicle-id": "SC1477473368403736",
    "position": 520,
    "role": "MedicalAssistance"
  }
}
```

Ejemplo de Mensaje 'CHECK_IN' (actualización de posición) en un Road Segment:

```
{
  "id": "MSG_147747746948753",
  "type": "TRAFFIC",
  "timestamp": 1477746948753,
  "msg": {
    "action": "CHECK_IN",
    "road": "R1",
    "road-segment": "R1S4a",
    "vehicle-id": "SC1477473368403736",
    "position": 580,
    "role": "MedicalAssistance"
  }
}
```

Ejemplo de Mensaje 'VEHICLE_OUT' al salir de un Road Segment:

```
{
  "id": "MSG_1477489498731",
  "type": "TRAFFIC",
  "timestamp": 1477489498731,
  "msg": {
    "action": "VEHICLE_OUT",
    "road": "R1",
    "road-segment": "R1S4a",
    "vehicle-id": "SC1477473368403736",
    "position": 640,
    "role": "MedicalAssistance"
  }
}
```

4.2.4 SE PUBLICAN ALERTAS EN LAS CARRETERAS

Los vehículos reportan en el topic

```
<topic-base>/road/{id-segmento}/alerts
```

cuando les sobreviene algún problema (un INCIDENT relacionado con o una situación de peligro, o un TRAFFIC_ACCIDENT en caso de accidente individual o múltiple). El vehículo reportará un mensaje de tipo M2 (ya descrito en los incidentes de carretera).

Pero, a diferencia del canal 'info' (donde publican las carreteras), son los vehículos los que publican en el canal 'alerts', para indicar de algún problema. Las carreteras, al recibir una alerta, inmediatamente la retransmiten por el canal 'info' de dicha carretera/segmento, para que el resto de vehículos sea informado.

Ejemplo de mensaje M2. Alerta por accidente

Topic MQTT: <topic-base>/road/{id-segmento}/alerts

```
{
  "id": "MSG_1477472671831",
  "type": "ROAD_INCIDENT",
  "timestamp": 1477472671831,
  "msg": {
    "rt": "traffic::alert",
    "incident-type": "TRAFFIC_ACCIDENT",
    "id": "7129632011",
    "road": "R2",
    "road-segment": "R2S1",
    "starting-position": 10,
    "ending-position": 10,
    "description": "Vehicle Crash",
    "status": "Active",
    "link": "/incident/7129632011"
  }
}
```

5 EJERCICIOS A RESOLVER

Durante la sesión presencial, se plantearán ejercicios prediseñados, y se diseñarán nuevos escenarios (a discutir con los alumnos). Todos estos escenarios se pueden desplegar en vuestras Raspberry Pi asociadas.

Para desarrollar estos ejercicios se tomará como base el código del proyecto smartcar (disponible en PoliformaT > Recursos > S2 > Código Lab 3 - SmartTraffic - Ejercicios).

Ejercicio 5.1) Configura el vehículo para que inicie en algún punto del PTPaterna y que reporte su ubicación. Comprobar como el segmento de carretera incrementa el número de vehículos existentes en el tramo. Al detener el vehículo, reportará que sale de dicho segmento de carretera.

Ejercicio 5.2) Inicializa varios vehículos (tantos como sea necesarios) y que entren en un mismo tramo, hasta conseguir cambiar su estado de congestión.

Ejercicio 5.3) Modifica el proyecto smartcar para crear otro nuevo dispositivo de tipo 'señal de tráfico - límite de velocidad'. Ubica dicha señal en un tramo en el que haya vehículos, y reporta un límite de velocidad inferior al existente en la vía.

Ejercicio 5.4) Ubica un vehículo el mismo segmento que se ha generado la señal de tráfico - límite de velocidad. Queremos que cuando esta señal vuelva a reportar un valor, que el vehículo lo reciba y 'simule' que modifica su velocidad (muestre por pantalla un mensaje indicando que reduce/aumenta su velocidad). Para ello deberá establecer su velocidad como la mínima entre la velocidad que indica la señal de límite de velocidad, y la velocidad actual de la vía³.

Ejercicio 5.5) Haz que un vehículo reporte un accidente en un tramo. Otros vehículos que estén circulando por el mismo tramo deben recibir el mensaje (que retransmitirá el servicio de carretera, esto no tenéis que implementarlo) a través del canal 'info'.

³ Recordad que esto se puede consultar a través de una llamada REST al segmento de carretera

6 EJERCICIO: SMART TRAFFIC REDUCIDO

6.1 SERVICIO DE SMART TRAFFIC REDUCIDO

Si has llegado aquí es que no has podido desarrollar sobre el escenario de Smart Traffic que se ha descrito con anterioridad. Por lo que, en este caso, desarrollarás tú una pequeña infraestructura que ofrezca una funcionalidad simplificada de lo que ofrece el servicio descrito.

Tomando como referente el escenario planteado en las secciones anteriores, vamos a ofrecer un pequeño servicio de tráfico orientado hacia la gestión de emergencias. Vas a tener que desarrollar 3 componentes (un vehículo, un gestor de carreteras, y una Smart City que jugará el papel de coordinador de una emergencia en carretera). Los mensajes que se intercambiarán en las comunicaciones entre los dispositivos están descritos en la figura inferior, y como se puede observar, son una simplificación de los mensajes (M1 a M4) utilizados por el servicio de Smart Traffic.

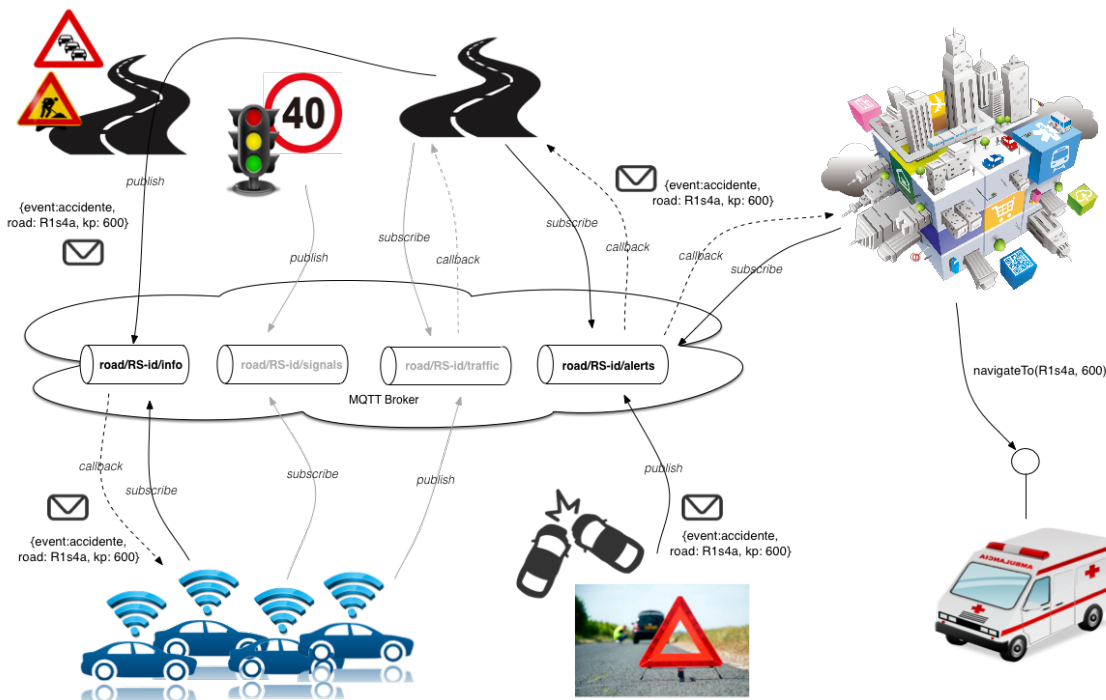


Figura 4 : Esquema de comunicaciones del escenario SmartTraffic Reducido

El funcionamiento de la solución que se propone desarrollar es la siguiente:

Paso 1) Al iniciar el sistema, todos los participantes (servicios y dispositivos IoT) se suscriben a sus topics de interés. De esta manera:

- **Red de carreteras:** cada segmento de la red se suscribe a su canal `alerts`
- **Vehículos:** cada vez que un vehículo entra en un segmento de carretera, se suscribe al canal de `info` de dicho segmento. Cada vez que sale de un segmento, cancela su suscripción a este canal.
- **Smart City:** se suscribirá a todos los canales de `alerts` de las carreteras que estén bajo su gestión (tal y como ocurre en el mundo real)

No es obligatorio que los vehículos se suscriban a los canales de `alerts`. Si no lo hacen (por ejemplo, cuando están detenidos), no recibirán la información que por allí se transmite.

Paso 2) El vehículo sufrirá un accidente (al iniciar, o al poco tiempo) y reportará un mensaje por el canal `alerts` al segmento por el que esté circulando. Tanto la carretera como la smartcity recibirán dicho mensaje que deberán gestionar:

Paso 3)

a) La carretera retransmitirá por el canal `info` el mensaje recibido por el canal `alerts`, informando al resto de vehículos.

b) La SmartCity enviará una ambulancia al lugar del accidente.

Paso 4) Los vehículos que circulan por el mismo segmento recibirán por el canal de `info` información sobre el accidente.

A continuación vamos a realizar una serie de ejercicios guiados para resolver cada uno de los pasos.

A partir de este momento, y para evitar colisiones con el Servicio de Tráfico, cambiaréis vuestro 'topic-base' para que, en vez de utilizar:

```
es/upv/pros/tatami/smartcities/traffic/PTPaterna/road/{id-seg}/<canal>
```

utilizaréis la siguiente estructura⁴, donde `<v-id>` será un 'id vuestro' (p.e. `jjfons`):

```
iot/2023/<v-id>/road/{id-segmento}/<canal>  
iot/2023/jjfons/road/R1s1/alerts
```

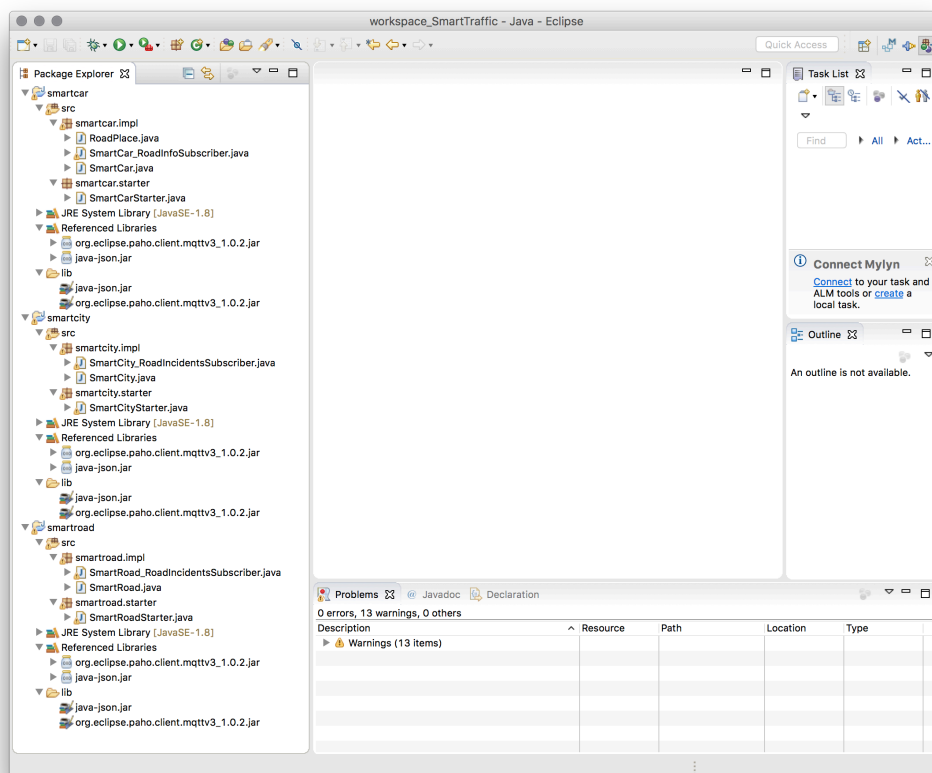
⁴ Deberéis cambiar/adaptar el código que os paso para hacer esto

6.2 EJERCICIO PASO 1. INICIAR LA SOLUCIÓN ...

Necesitamos 3 proyectos Java: **smartcity**, **smartroad** y **smartcar**⁵.

El primero representará a la SmartCity (que se encargará de gestionar accidentes), el segundo a un road/segment (le indicaremos que es segmento de carretera en concreto, por ejemplo R5s1) y un el tercero lo usaremos para controlar un vehículo que al iniciar sufrirá un accidente.

Desde PoliformaT, descarga estos proyectos. Puedes abrirlos en VSCode, o importarlos a Eclipse (Import ... > Projects From Folder or Archive).



Para funcionar requieren los componentes para gestionar comunicaciones MQTT librería *MQTT/Paho*⁶, *org.eclipse.paho.client.mqttv3_1.0.2.jar*), y JSON (librería *java-json.org*). Hay una carpeta (lib) dentro de cada uno de los proyectos con estas librerías previamente añadidas al Build Path.

⁵ Están disponibles en PoliformaT > Planificación > Sesión 2 y en PoliformaT > Recursos > S2

⁶ <http://www.eclipse.org/paho/>

Si nos fijamos, cada proyecto contiene un 'starter' (Main) que permite ejecutar el proyecto (cada uno funciona por separado), y un conjunto de clases de implementación: el recurso en sí (SmartCar, SmartRoad y SmartCity), y un suscriptor en MQTT. Analizar el código de cada uno de estas clases, fijándose en los `starters` y en las operaciones de 'subscribe' de los `subscribers`. Nos daremos cuenta que al arrancar, cada uno de estos recursos se suscribe a su topic de interés. Comprobar la línea que especifica el broker al que se conectará:

```
static final String BROKER_URL = "tcp://tambori.dsic.upv.es:10083";
```

Si analizamos, por ejemplo, el proyecto smartcity, encontraremos los recursos:

- SmartCity: clase que representa a una ciudad inteligente. Posee un identificador de ciudad.
- SmartCity_RoadIncidentsSubscriber: clase que usa la SmartCity para conectarse y suscribirse a topics de interés (en este caso, a la carretera R5s1).

```
public class SmartCity_RoadIncidentsSubscriber implements MqttCallback
```

Toda clase que envíe (rol publisher) notificaciones o las reciba (rol suscriber) debe implementar la interfaz `MqttCallback`. Esta interfaz requiere que se implementen los siguientes métodos (que serán invocados usando el patrón inversión de control):

- `public void connectionLost(Throwable t)` : es llamada cuando se pierde la conexión con el broker MQTT
- `public void deliveryComplete(IMqttDeliveryToken token)` : es llamada cuando se ha terminado de subir un mensaje al topic enviado
- `public void messageArrived(String topic, MqttMessage msg) throws Exception`: es llamada cuando se recibe un mensaje, al estar suscrito a un topic del broker MQTT

En la clase `SmartCity_RoadIncidentsSubscriber`, el método `messageArrived` de momento sólo muestra por pantalla el mensaje recibido. Según la descripción, debería revisar el mensaje, y si el tipo de evento es de tipo 'accidente', debería iniciar el proceso de atención al siniestro.

```
public void messageArrived(String topic, MqttMessage message) throws Exception
{
    System.out.println("-----");
    System.out.println("| Topic:" + topic);
    System.out.println("| Message: " + new String(message.getPayload()));
    System.out.println("-----");

    // DO SOME MAGIC HERE!
}
```

Además de estas operaciones, se han implementado las siguientes:

```
public void connect() {
    // setup MQTT Client
    String clientId = this.id; // id del cliente MQTT
    connOpt = new MqttConnectOptions();
    connOpt.setCleanSession(true);
    connOpt.setKeepAliveInterval(30);
    // connOpt.setUserName(M2MIO_USERNAME);
    // connOpt.setPassword(M2MIO_PASSWORD_MD5.toCharArray());

    // Connect to Broker
    try {
        myClient = new MqttClient(BROKER_URL, clientId);
        myClient.setCallback(this);
        myClient.connect(connOpt);
    } catch (MqttException e) {
        e.printStackTrace();
        System.exit(-1);
    }
    this._debug("Connected to " + BROKER_URL);
}
```

Donde:

- **BROKER_URL**: define la URL del broker MQTT
- **MqttClient myClient**: cliente Java para conectarse a MQTT (parte de la implementación de MQTT/Paho)
- **MqttConnectOptions connOpt**: configuración de conexión, incluyendo opciones sobre *keep-alive*, usuario y password (si se usa identificación para acceder al broker), entre otras.

Para desconectar a un cliente del broker (y eliminar todas sus suscripciones), se ejecuta la operación:

```
myClient.disconnect();
```

Además de las operaciones que requiere MqttCallback, se define una operación para suscribirse a un topic:

```
public void subscribe(String myTopic) {

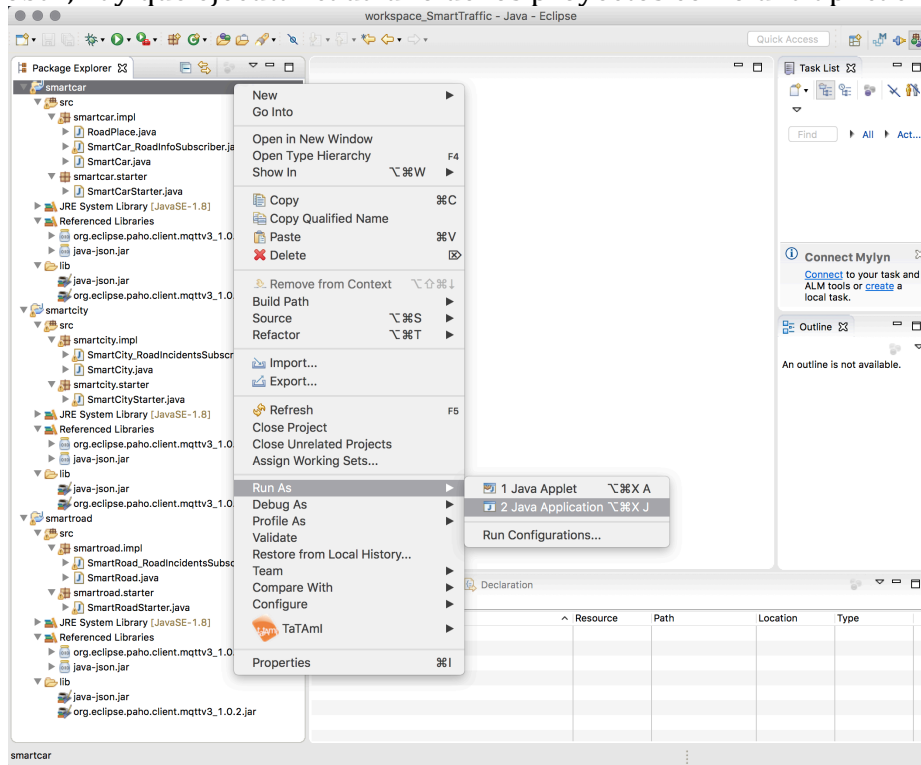
    // subscribe to topic
    try {
        int subQoS = 0;
        myClient.subscribe(myTopic, subQoS);
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Y otra para cancelar una suscripción a un topic:

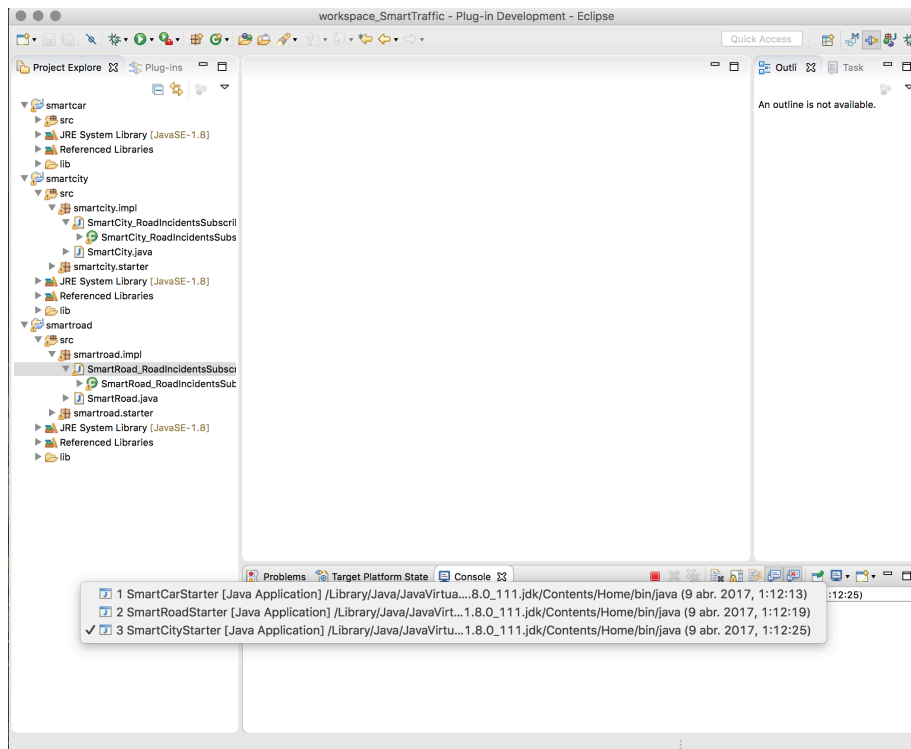
```
public void unsubscribe(String myTopic) {

    // unsubscribe from topic
    try {
        myClient.unsubscribe(myTopic);
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Para probar, hay que ejecutar cada uno de los proyectos como una aplicación Java.



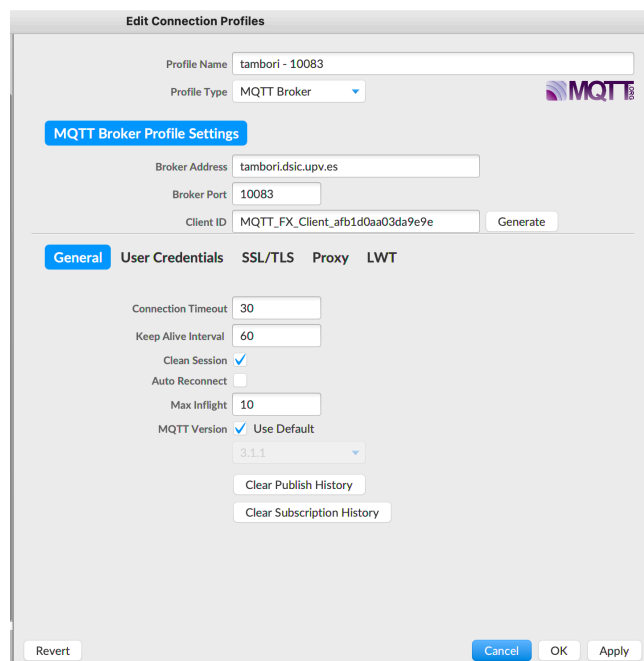
Una vez ejecutados, tendremos 3 consolas de ejecución:



Para probar que funciona, podemos publicar un mensaje usando el cliente **MQTT.fx**⁷. Abridlo y configurad un nuevo perfil de conexión (⚙️) con el broker MQTT utilizado.

Aseguraos de modificar el valor del campo 'Client ID' para que no haya dos usuarios con el mismo identificador de cliente (darle al Generate).

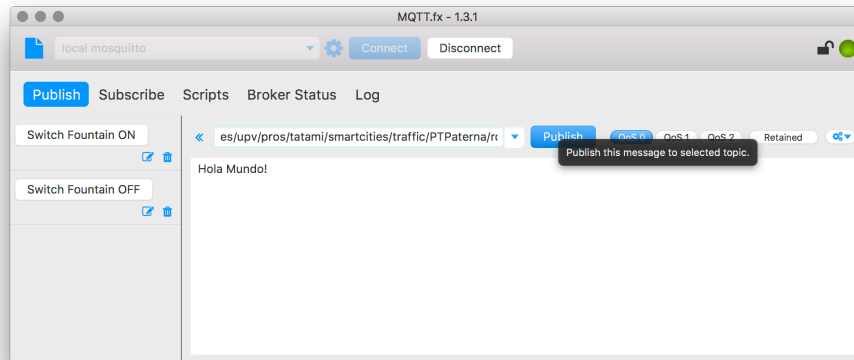
Una vez creado el perfil, os conectáis al servidor.



⁷ [Descargar MQTT.fx 1.7.1 desde PoliformaT > Recursos > Software > MQTT.fx](#)

Ahora vais a la pestaña 'Publish', indicáis un **topic** y mandáis un mensaje cualquiera (en estos momentos reconocerá cualquier cosa). Probad a enviar a los topics de info y de alerts.

```
iot/2023/<v-id>/road/R5s1/info  
iot/2023/<v-id>//road/R5s1/alerts
```



Comprobad qué ocurre en las consolas de Java cada vez que enviáis un mensaje a cada uno de estos topics. ¿Quién debería atender a cada topic?



6.3 EJERCICIO PASO 2. NOTIFICAR ACCIDENTE POR EL CANAL 'ALERTS'

Al sufrir un accidente, el vehículo enviará un mensaje por el canal `alerts` al segmento por el que esté circulando, indicando también el punto (kilométrico) donde ha ocurrido.

En este paso trabajaremos en el proyecto **SmartCar**. Debemos añadir una clase (que extenderá `MqttCallback`) al proyecto para realizar la notificación. En PoliformaT encontraréis la clase `SmartCar_IncidentNotifier.java` en el Paso 2. Copiadla dentro del paquete `smartcar.impl`.

```
public class SmartCar_IncidentNotifier implements MqttCallback
```

Además de las operaciones que requiere `MqttCallback`, se define una operación para publicar un mensaje en un topic:

```
public void alert(String smartCarID, String notificatType, RoadPlace place) {
    String myTopic = "iot/2023/<v-id>/road/"
        + place.getRoad() + "/info";
    MqttTopic topic = myClient.getTopic(myTopic);

    // publish incident
    JSONObject pubMsg = new JSONObject();
    try {
        pubMsg.put("vehicle", smartCarID);
        pubMsg.put("event", notificatType);
        pubMsg.put("road", place.getRoad());
        pubMsg.put("kp", place.getKm());
    } catch (JSONException e1) {
        return;
    }

    int pubQoS = 0;
    MqttMessage message = new MqttMessage(pubMsg.toString().getBytes());
    message.setQos(pubQoS);
    message.setRetained(false);

    // Publish the message
    MqttDeliveryToken token = null;
    try {
        // publish message to broker
        token = topic.publish(message);
        this._debug(pubMsg.toString());
        token.waitForCompletion();
        Thread.sleep(100);
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Donde:

- `MqttTopic topic` : permite seleccionar un topic, consultando al cliente `MqttClient`
- `MqttMessage message` : mensaje que se enviará al topic (en este caso, en formato JSON)
- `MqttDeliveryToken token` : permite conocer el estado de la publicación en un topic (`topic.publish(message)`) y forzar a esperar que se complete el envío antes de seguir (`token.waitForCompletion()`);

Reemplazad el fichero `smartcar.impl.SmartCar.java` por el que encontraréis en el Paso 2.

```
public class SmartCar {  
  
    protected String smartCarID = null;  
    protected RoadPlace rp = null;  
    protected SmartCar_RoadInfoSubscriber subscriber = null;  
    protected SmartCar_IncidentNotifier notifier = null;  
  
    public SmartCar(String id) {  
  
        this.setSmartCarID(id);  
  
        this.rp = new RoadPlace("R5s1", 10);  
  
        this.subscriber = new SmartCar_RoadInfoSubscriber(this);  
        this.subscriber.connect();  
        this.subscriber.subscribe("iot/2023/<v-id>/road/R5s1/info");  
  
        this.notifier = new SmartCar_IncidentNotifier(id);  
        this.notifier.connect();  
  
    }  
}
```

... y además implementa un nuevo método `notifyIncident()` que notifique la alerta.

```
public void notifyIncident(String incidentType) {  
    if ( this.notifier == null )  
        return;  
  
    this.notifier.alert(this.getSmartCarID(), incidentType,  
        this.getCurrentPlace());  
}
```

Por último, vamos al **starter** y le indicamos que provoque un incidente, añadiendo la siguiente línea de código:

```
public class SmartCarStarter {

    public static void main(String[] args) {

        SmartCar sc1 = new SmartCar("SmartCar0XX");

        try {
            Thread.sleep(5000);
        } catch (InterruptedException e) {
        }

        sc1.changeRoad("R5s1", 100);
        sc1.notifyIncident("accidente");

    }

}
```

Para probar que funciona, ahora ejecutamos primero los proyectos SmartCity y SmartRoad (que se suscriben al canal de alert de R5s1). Al iniciar después el SmartCar, recibirán el mensaje y lo mostrarán por la consola.

```
SmartCarStarter [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_111.jdk/Contents/Home/bin/java (9 abr. 2017, 2:09:38)
(SmartCar: SmartCar001) Connected to tcp://localhost:1883
(SmartCar: SmartCar001) Publishing to topic "es/upv/pros/tatami/smartcities/traffic/PTPaterna/road/R5s1/alerts"
(SmartCar: SmartCar001) {"road":"R5s1","kp":100,"event":"accidente","vehicle":"SmartCar001"}

SmartRoadStarter [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_111.jdk/Contents/Home/bin/java (9 abr. 2017, 2:09:32)
(SmartRoad: R5s1) Connected to tcp://localhost:1883
-----
| Topic:es/upv/pros/tatami/smartcities/traffic/PTPaterna/road/R5s1/alerts
| Message: {"road":"R5s1","kp":100,"event":"accidente","vehicle":"SmartCar001"}
-----

SmartCityStarter [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_111.jdk/Contents/Home/bin/java (9 abr. 2017, 2:09:35)
(SmartCity: VLC.net) Connected to tcp://tambori.dsic.upv.es:1883
-----
| Topic:es/upv/pros/tatami/smartcities/traffic/PTPaterna/road/R5s1/alerts
| Message: {"road":"R5s1","kp":100,"event":"accidente","vehicle":"SmartCar001"}
-----
```

6.4 EJERCICIO PASO 3A. LA CARRETERA RETRANSMITE ALERTA ...

La carretera retransmite por el canal de información el mensaje recibido por el canal de alerts.

En este paso trabajaremos en el proyecto SmartRoad. Crearemos una clase que se encargue de realizar la notificación (SmartRoad_IncidentNotifier.java), disponible en el código del Paso3a (PoliformaT). Proporciona el método `notify()`.

```
public void notify(String infoMessage) {

    String myTopic = "iot/2023/v-id/road/" + this.road.id + "/info";
    MqttTopic topic = myClient.getTopic(myTopic);

    int pubQoS = 0;
    MqttMessage message = new MqttMessage(infoMessage.getBytes());
    message.setQos(pubQoS);
    message.setRetained(false);

    // Publish the message
    this._debug("Publishing to topic \"" + topic + "\" qos " + pubQoS);
    MqttDeliveryToken token = null;
    try {
        // publish message to broker
        token = topic.publish(message);
        this._debug(infoMessage);
        // Wait until the message has been delivered to the broker
        token.waitForCompletion();
        Thread.sleep(100);
    } catch (Exception e) {
        e.printStackTrace();
    }

}
```

Hacemos que la clase SmartRoad inicialice este notificador:

```
public class SmartRoad {

    protected SmartRoad_IncidentNotifier notifier = null;
    protected SmartRoad_RoadIncidentsSubscriber subscriber = null;
    protected String id = null;

    public SmartRoad(String id) {
        this.setId(id);
        this.subscriber = new SmartRoad_RoadIncidentsSubscriber(this);
        this.subscriber.connect();
        this.subscriber.subscribe("iot/2023/<v-id>/road/" + id + "/alerts");

        notifier = new SmartRoad_IncidentNotifier(this);
        notifier.connect();
    }

    public String getId() {
        return id;
    }

    public void setId(String id) {
        this.id = id;
    }

    public void notify(String message) {
        this.notifier.notify(message);
    }

}
```

Y modificamos el método messageArrived() en el suscriptor para redirigir el mensaje al topic info, usando el método notify() de la clase SmartRoad_IncidentNotifier.java.

```
public void messageArrived(String topic, MqttMessage message) throws Exception
{
    String payload = new String(message.getPayload());

    System.out.println("-----");
    System.out.println("| Topic:" + topic);
    System.out.println("| Message: " + payload);
    System.out.println("-----");

    // DO SOME MAGIC HERE!
    this.road.notify(payload);
}
```

6.5 EJERCICIO PASO 3B. LA SMARTCITY ENVÍA UNA AMBULANCIA ...

En este ejercicio modificaremos el proyecto SmartCity, para que envíe una ambulancia al lugar del accidente. Vamos a modificar el método `messageArrived()` del `SmartCity_RoadIncidentsSubscriber` para pedirle a la ambulancia (`SmartCar021` - `ttmi021.iot.upv.es`) que se ponga en marcha:

```
@Override
public void messageArrived(String topic, MqttMessage message) throws Exception {

    String payload = new String(message.getPayload());
    System.out.println("-----");
    System.out.println("| Topic: " + topic);
    System.out.println("| Message: " + payload);
    System.out.println("-----");

    // ttmi021 jugará el rol de ambulancia
    String url = "http://ttmi021.iot.upv.es:8182/car/engine";
    String putPayload = "{action:forward}";

    try{

        // Vamos a invocar a un recurso a través de su API REST
        // ClientResource (del framework Restlet) nos asiste a generar
        // invocaciones HTTP/REST
        // - debemos indicarle la URI destino
        // - podemos hacer peticiones GET, POST, PUT, DELETE, ...
        // - debemos construir el mensaje/Payload a enviar
        // (en operaciones POST/PUT) a través de una Representation
        // - como resultado, devuelve una Representation

        ClientResource cr = new ClientResource(url);
        Representation payload = new StringRepresentation(putPayload,
                                                         MediaType.TEXT_PLAIN);
        Representation result = cr.put(payload);

    }catch(Exception e){
        e.printStackTrace();
    }

}
```

6.6 PASO 4A) LOS VEHÍCULOS RECIBEN ALERTA ...

Los vehículos que circulan por el mismo segmento recibirán por el canal de `info` información sobre el accidente.

Comprobad que el vehículo recibe la notificación vía el método `messageArrived()` en `SmartCar_RoadInfoSubscriber`.

6.7 EJERCICIO OPCIONAL: RUTAS

Este ejercicio tiene un coste relativamente elevado para su desarrollo. Realizar sólo en caso de querer profundizar en la interacción con la red de carreteras, y si se tiene tiempo disponible. En tal caso, reutilizar/integrar en el proyecto final de la asignatura (se valorará).

Implementar una pequeña 'ruta' para el vehículo. El vehículo deberá seguir los `RoadSegments` indicados en la ruta, cambiando de segmento en el punto que se le indique. Cada vez que entre o salga de un segmento, deberá informar a la red de carreteras a través del canal `traffic`.

Sobre el mapa de PTPaterna, simular que el vehículo empieza en (R5s1, 100) y termina en (R1s4a, 600).

* Recomendación: simplificad los cálculos asumiendo que la velocidad que se indica al vehículo es en Km/h. Así, si se le establece la velocidad de 100Km/h al motor (`{action:forward, speed:100}`), cada segundo recorre 30 metros (en realidad es algo menos). Por lo que, controlando 'el tiempo', podemos 'estimar' (aproximadamente) el avance (en metros) que realiza el vehículo, y cuándo termina el segmento por el que circula y entra (a través de una intersección) con el siguiente segmento de la ruta.

7 ANEXO - SIMULADOR SMARTTRAFFIC

Se ha desarrollado una infraestructura de soporte a la simulación, de manera que cada alumno/grupo pueda disponer de un simulador operativo para realizar pruebas, o desarrollar sus proyectos.

Este simulador, junto a la infraestructura de soporte se ejecuta junto al servicio de tráfico, y puede estar disponible en:

ttmi008.iot.upv.es:8182
ó
tambori.dsic.upv.es:10082

Por defecto, el simulador no carga ningún modelo de vehículos ni de incidencias. Esto significa que al crear el simulador, nadie circula por él, ni se generan 'incidencias'. En la sección 'Configuración del Simulador' se muestra cómo crear elementos de simulación (aleatorios o fijos) para que 'vivan' dentro del simulador.

Si fuera necesario, se podrían configurar diferentes simuladores con diferentes 'contextos'. Se ha decidido seguir esta estrategia para que se pueda configurar el escenario, si necesita o no la creación de vehículos aleatorios (que aparecen, siguen una ruta aleatoria en el simulador y desaparecen), si necesitan tener una flota de vehículos fijos (a los que asignar rutas y que no desaparezcan del simulador), y si se desea o no que se generen incidencias (calles cortadas, obras, accidentes, etc.) aleatoriamente.

Por ejemplo, para el simulador base PTPaterna, si queremos suscribirnos al canal de información del segmento R12S1, deberemos suscribirnos a la siguiente cola:

es/upv/pros/tatami/smartcities/traffic/PTPaterna/road/R12S1/info

El simulador atiende a los canales/colas/topics de comunicación descritos en este documento, por lo que, en el caso del simulador 'PTPaterna' existirán los siguientes topics (por ejemplo, sobre el segment R1S1):

- Canal de información:
es/upv/pros/tatami/smartcities/traffic/PTPaterna/road/R1S1/info
- Canal de notificaciones de entrada/salida/check-in de vehículos
es/upv/pros/tatami/smartcities/traffic/PTPaterna/road/R1S1/traffic

- Canal de actualización de señales de tráfico
`es/upv/pros/tatami/smartcities/traffic/PTPaterna/road/R1S1/signals`
- Canal para notificación de alertas
`es/upv/pros/tatami/smartcities/traffic/PTPaterna/road/R1S1/alerts`

7.1 INTERACTUAR CON EL SIMULADOR VÍA REST

En el caso del simulador Intersección recién creado, para interactuar con él, según algunas cosas ya vistas en la sección 'APIs REST', y atendiendo al puerto 8182, podremos realizar las siguientes consultas realizadas con un cliente REST (Postman, RESTClient, etc.).

7.1.1 CONSULTAR EL ESTADO DEL SIMULADOR

Para comprobar rápidamente si el simulador está operativo, y obtener su id y número de paso de simulación actual, se ha habilitado la siguiente URL:

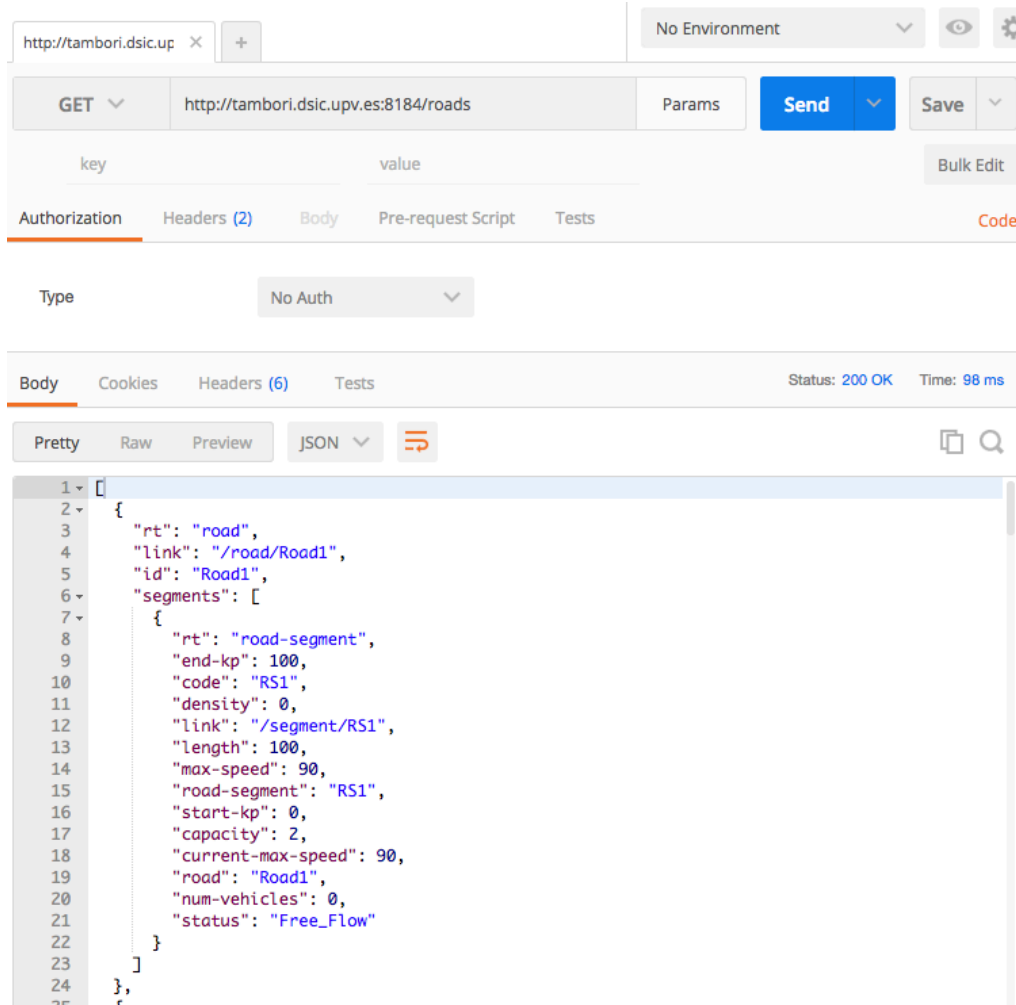
GET, <http://ttmi008.iot.upv.es:8182/simulator>

Ejemplo de resultado:

```
{"step":1612,"id":"PTPaterna"}
```

7.1.2 ¿QUÉ CARRETERAS TIENE NUESTRO SIMULADOR?

GET, <http://ttmi008.iot.upv.es:8182/roads>



The screenshot shows a REST client interface with the following details:

- Request:** GET `http://tambori.dsic.upv.es:8184/roads`
- Response Status:** 200 OK, Time: 98 ms
- Response Body (JSON):**

```

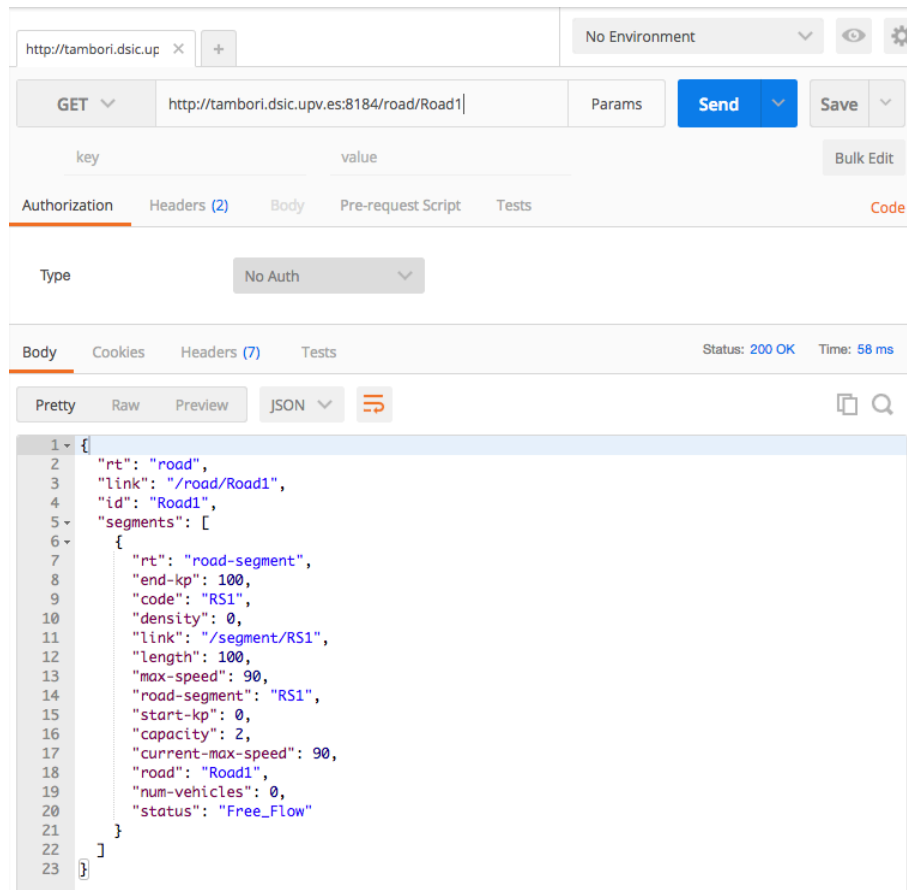
1  {
2    "rt": "road",
3    "link": "/road/Road1",
4    "id": "Road1",
5    "segments": [
6      {
7        "rt": "road-segment",
8        "end-kp": 100,
9        "code": "RS1",
10       "density": 0,
11       "link": "/segment/RS1",
12       "length": 100,
13       "max-speed": 90,
14       "road-segment": "RS1",
15       "start-kp": 0,
16       "capacity": 2,
17       "current-max-speed": 90,
18       "road": "Road1",
19       "num-vehicles": 0,
20       "status": "Free_Flow"
21     }
22   ]
23 }
24 },
25 ]

```

Notad que se está dirigiendo la petición al puerto 8182 (según indica nuestro simulador).

7.1.3 OBTENER INFORMACIÓN SOBRE UNA CARRETERA

GET, <http://ttmi008.iot.upv.es:8182/road/Road1>



The screenshot shows a REST client interface with the following details:

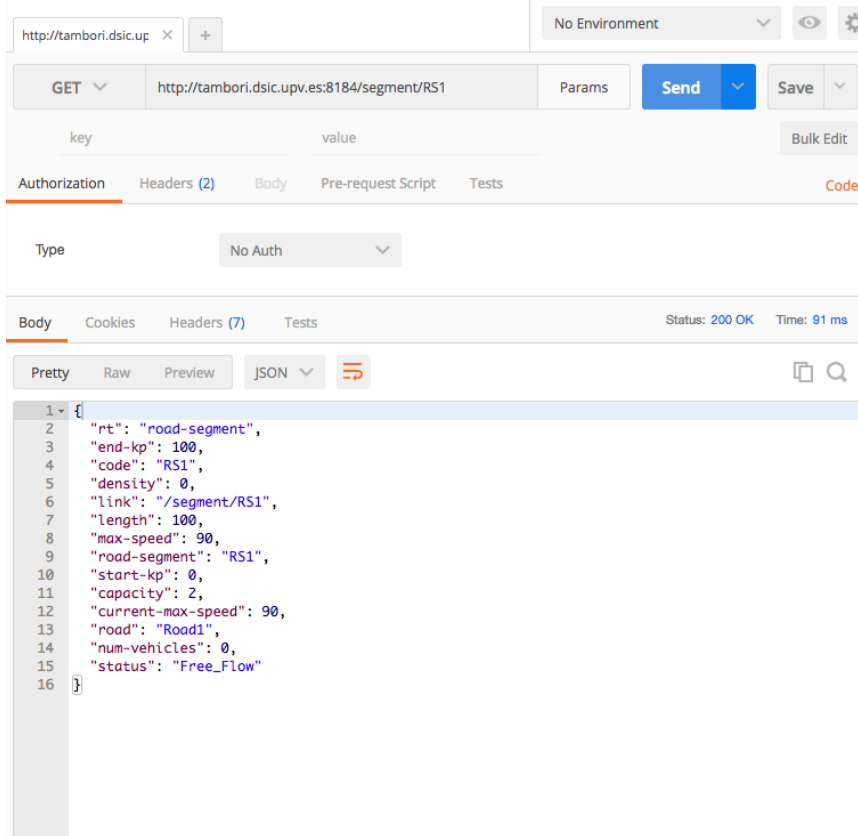
- Request:** GET `http://ttmi008.iot.upv.es:8182/road/Road1`
- Response:** Status: 200 OK, Time: 58 ms
- Body (JSON):**

```

1 {
2   "rt": "road",
3   "link": "/road/Road1",
4   "id": "Road1",
5   "segments": [
6     {
7       "rt": "road-segment",
8       "end-kp": 100,
9       "code": "RS1",
10      "density": 0,
11      "link": "/segment/RS1",
12      "length": 100,
13      "max-speed": 90,
14      "road-segment": "RS1",
15      "start-kp": 0,
16      "capacity": 2,
17      "current-max-speed": 90,
18      "road": "Road1",
19      "num-vehicles": 0,
20      "status": "Free_Flow"
21    }
22  ]
23 }
```

7.1.4 OBTENER INFORMACIÓN SOBRE UN SEGMENTO DE CARRETERA

GET, <http://ttmi008.iot.upv.es:8182/segment/RS1>



The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://ttmi008.iot.upv.es:8182/segment/RS1
- Status:** 200 OK
- Time:** 91 ms
- Response Body (JSON):**

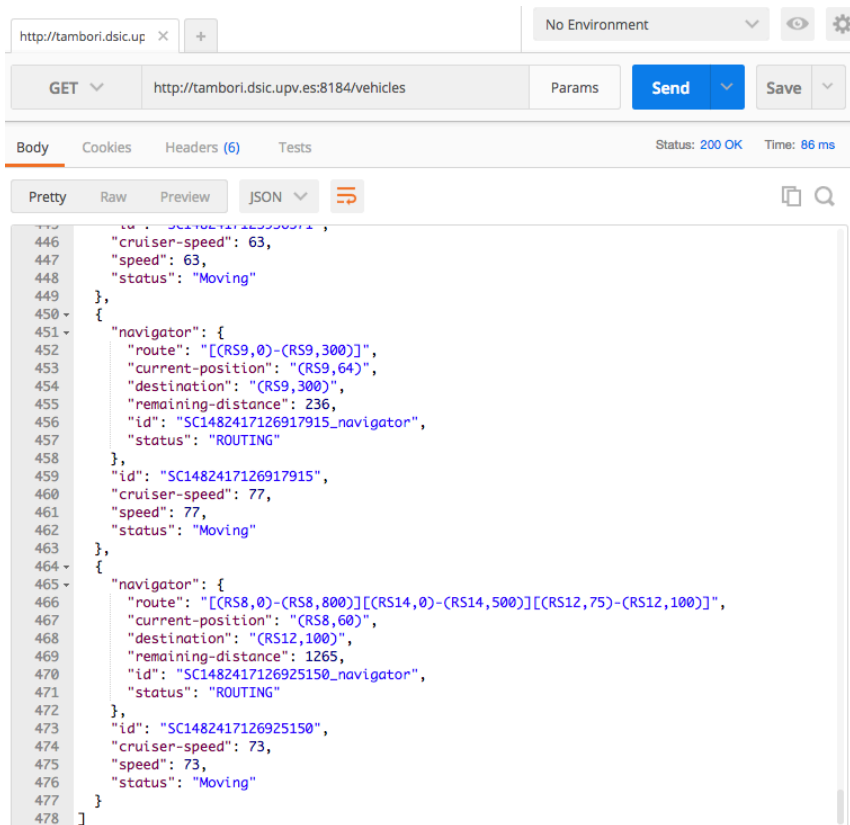
```
{
  "rt": "road-segment",
  "end-kp": 100,
  "code": "RS1",
  "density": 0,
  "link": "/segment/RS1",
  "length": 100,
  "max-speed": 90,
  "road-segment": "RS1",
  "start-kp": 0,
  "capacity": 2,
  "current-max-speed": 90,
  "road": "Road1",
  "num-vehicles": 0,
  "status": "Free_Flow"
}
```

7.1.5 ¿QUÉ VEHÍCULOS CIRCULAN POR EL SIMULADOR?

GET, <http://ttmi008.iot.upv.es:8182/vehicles>

Ejemplo de respuesta:

```
[
  {
    "navigator": {
      "route": "<No route>",
      "current-position": "(RS3,35)",
      "destination": "(RS3,100)",
      "remaining-distance": 0,
      "id": "SC14825374963259646_navigator",
      "status": "REACHED_DESTINATION",
      "id": "SC14825374963259646",
      "characterization": {
        "role": "PublicTransport",
        "type": "Bus"
      },
      "cruiser-speed": 84,
      "speed": 0,
      "status": "Parked"
    },
    {
      "navigator": {
        "route": "<No route>",
        "current-position": "(RS12,35)",
        "destination": "(RS12,100)",
        "remaining-distance": 0,
        "id": "SC14825376013182545_navigator",
        "status": "REACHED_DESTINATION",
        "id": "SC14825376013182545",
        "characterization": {
          "role": "PrivateUsage",
          "type": "Automobile"
        },
        "cruiser-speed": 85,
        "speed": 0,
        "status": "Parked"
      },
      {
        "navigator": {
          "route": "[ (RS6,0) - (RS6,250) ] [ (RS4,550) - (RS4,1000) ] [ (RS8,500) - (RS8,800) ] [ (RS14,0) - (RS14,500) ] [ (RS12,75) - (RS12,200) ] [ (RS12,0) - (RS12,175) ] [ (RS10,0) - (RS10,600) ] [ (RS4,250) - (RS4,1000) ] [ (RS8,500) - (RS8,550) ] [ (RS9,0) - (RS9,700) ] [ (RS11,0) - (RS11,600) ] [ (RS12,0) - (RS12,100) ]",
          "current-position": "(RS6,62)",
          "destination": "(RS12,100)",
          "remaining-distance": 4538,
          "id": "SC14825376043173876_navigator",
          "status": "ROUTING",
          "id": "SC14825376043173876",
          "characterization": {
            "role": "Police",
            "type": "Automobile"
          },
          "cruiser-speed": 71,
          "speed": 71,
          "status": "Moving"
        }
      }
    }
  ]
```



7.1.6 OBTENER INFORMACIÓN SOBRE UN VEHÍCULO CONCRETO

GET, <http://ttmi008.iot.upv.es:8182/vehicle/3240JVM>

Ejemplo de respuesta:

```
{ "characterization": { "type": "Bus", "role": "PublicTransport" },  
  "navigator": { "current-position": "(RS14,109)", "destination": "(RS12,100)",  
    "remaining-distance": 416, "id": "SC1482020097479368_navigator",  
    "status": "ROUTING", "id": "SC1482020097479368", "cruiser-  
    speed": 101, "speed": 90, "status": "Moving" }
```

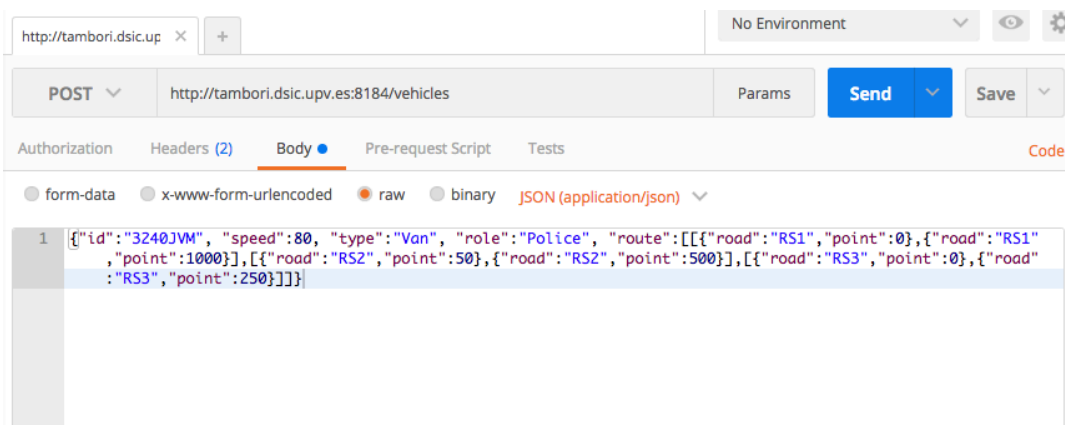
7.1.7 CREAR UN NUEVO VEHÍCULO EN EL SIMULADOR

POST, <http://ttmi008.iot.upv.es:8182/vehicles>

El siguiente ejemplo (payload) crea un Autobús (type) de Transporte Público (role) con identificador 3240JVM, con una velocidad 'de crucero' de 80 Km/h (velocidad a la que irá siempre que no limite la velocidad de la carretera/calle), y una ruta aproximada de 1700metros:

```
{ "id": "3240JVM", "speed": 80, "type": "Bus", "role": "PublicTransport",  
  "route": [ [ { "road": "RS1", "point": 0 }, { "road": "RS1", "point": 1000 }, [ { "road": "RS2",  
    "point": 50 }, { "road": "RS2", "point": 500 } ], [ { "road": "RS3", "point": 0 }, { "road": "RS  
3", "point": 250 } ] ] ] }
```

*NOTA1: Si no se indica 'type' y/o 'role', se asumen sus valores por defecto (type:PrivateUsage, role:Automobile).



También se puede generar un vehículo sin indicar su ruta, y establecerla después.

7.1.8 ESTABLECER UNA RUTA A UN VEHÍCULO EXISTENTE

PUT, <http://ttmi008.iot.upv.es:8182/vehicle/3240JVM>

El siguiente payload de ejemplo cambia la ruta al vehículo anterior:

```
{"route": [[{"road": "RS6", "point": 0}, {"road": "RS6", "point": 1000}], [{"road": "RS7", "point": 50}, {"road": "RS7", "point": 500}], [{"road": "RS8", "point": 0}, {"road": "RS8", "point": 250}]]}
```

*NOTA: recordar indicar las cabeceras "Content-Type": "application/json" y "Accept": "application/json"

7.1.9 ELIMINAR UN VEHÍCULO DEL SIMULADOR

DELETE, <http://ttmi008.iot.upv.es:8182/vehicle/3240JVM>

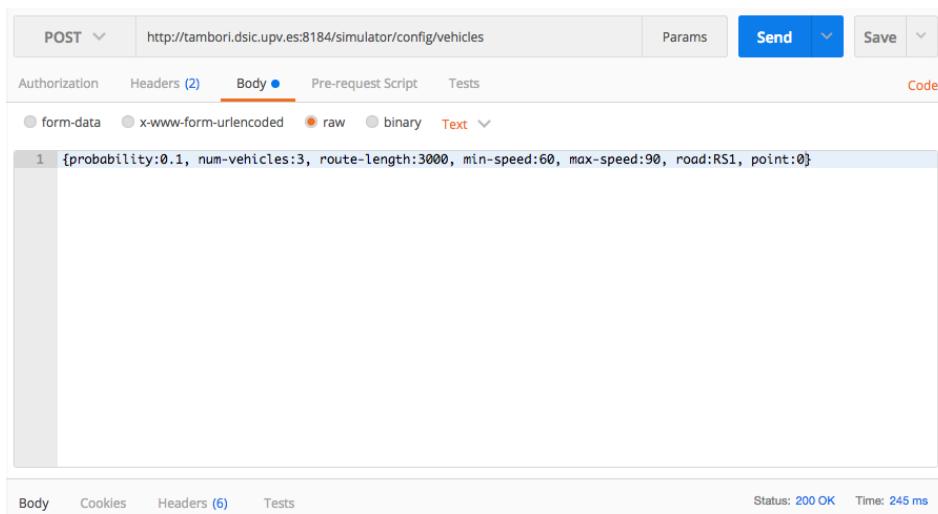
7.1.10 CONFIGURAR LA CREACIÓN DE VEHÍCULOS ALEATORIOS

Permite indicar al simulador que genere vehículos con una probabilidad, y que se les asigne una ruta de una longitud aproximada (si es posible).

El recurso sobre el que solicitar esta configuración es:

POST, <http://ttmi008.iot.upv.es:8182/simulator/config/vehicles>

En el siguiente ejemplo se le está indicando al simulador que genere, con una probabilidad de un 10%, entre 1 y 3 vehículos (aleatoriamente cada vez), con una longitud de ruta de 3000 m, una velocidad de cruce de entre 60 y 90 Km/h y que se origine en el segmento RS1 en el punto kilométrico 0.



La mayoría de parámetros para inicializar vehículos, o bien tienen un valor por defecto, o bien se generan automáticamente, en caso de no indicar explícitamente el valor del parámetro. Por ejemplo, en la llamada anterior, al no indicar tipo ni rol del vehículo, se generan por defecto Automóviles (*type:Automobile*) de uso privado (*role:PrivateUsage*). Por el contrario, si no se especificara una 'road' y un 'point' como origen de los vehículos, se inicializarían aleatoriamente por el mapa.

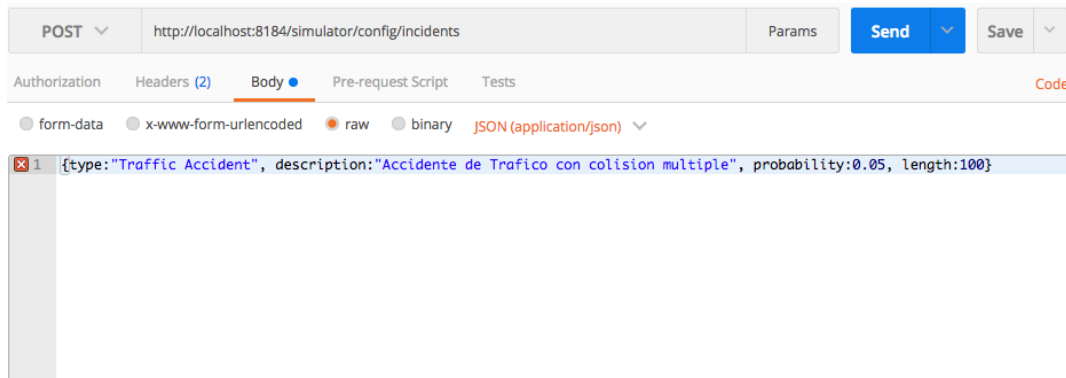
7.1.11 CONFIGURAR LA CREACIÓN DE INCIDENTES ALEATORIOS

Permite, de la misma manera, configurar la generación aleatoria de incidentes (obras, accidentes, calles cortadas, etc.) en cualquier road segment del mapa.

El recurso sobre el que solicitar esta configuración es:

POST, <http://ttmi008.iot.upv.es:8182/simulator/config/incidents>

En el siguiente ejemplo se le está indicando al simulador que genere, con una probabilidad de un 5% (en cada paso de simulación), un incidente de tipo "Traffic Accident" con una longitud de 100 m (si no se indica longitud, se genera un incidente en un punto).



POST http://localhost:8184/simulator/config/incidents Params Send Save

Authorization Headers (2) Body Pre-request Script Tests Code

form-data x-www-form-urlencoded raw binary JSON (application/json)

```
{type:"Traffic Accident", description:"Accidente de Trafico con colision multiple", probability:0.05, length:100}
```

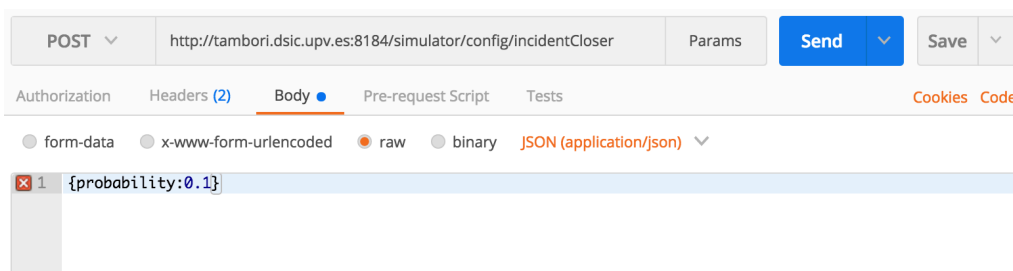
Los tipos de incidentes reconocidos por el simulador son:
"Traffic Accident", "Incident", "Traffic Congestion", "Working area", "Other incident"

7.1.12 CONFIGURAR LA RESOLUCIÓN (ELIMINACIÓN) ALEATORIA DE INCIDENTES

Permite cerrar (resolver) incidentes de manera aleatoria. El recurso sobre el que solicitar esta configuración es:

POST, <http://ttmi008.iot.upv.es:8182/simulator/config/incidentCloser>

En el siguiente ejemplo se cierra un incidente con una probabilidad del 10% (en cada paso de simulación).



7.1.13 CONSULTAR LOS TIPOS Y ROLES DE VEHÍCULOS PERMITIDOS

En el siguiente recurso se pueden consultar los tipos y roles de vehículos que reconoce el simulador.

GET, <http://ttmi008.iot.upv.es:8182/simulator/vehicles/characterization>

Ejemplo de resultado:

```
{ "types": ["Unclassified", "Airplane", "Helicopter", "Hovercraft", "Ship",  
  "Submarine", "Train", "Boxcar", "Caboose", "Snowmobile", "FarmTractor",  
  "Automobile", "Van", "Bus", "Truck", "Trailer", "Semitrailer", "Motorcycle",  
  "Bicycle", "Moped"],  
  "roles": ["Unknown", "PrivateUsage", "PublicTransport", "MedicalAssistance",  
  "Police", "Military", "Firetruck", "GoodsDelivery", "PublicServices"] }
```